

Universidade de Brasília – UnB
Faculdade UnB Gama – FCTE
Software Engineering

AWS Cloud Adoption Assistance Using Large Language Models

Author: Lucas Medeiros Confortin Rosa
Supervisor: Dr. Fabrício Ataides Braz

Brasília, DF
2025



Lucas Medeiros Confortin Rosa

AWS Cloud Adoption Assistance Using Large Language Models

Thesis submitted to the Undergraduate Program in Software Engineering at the Universidade de Brasília, as a partial requirement for obtaining the degree of Bachelor in Software Engineering.

Universidade de Brasília – UnB

Faculdade UnB Gama – FCTE

Supervisor: Dr. Fabrício Ataides Braz

Brasília, DF

2025

Lucas Medeiros Confortin Rosa
AWS Cloud Adoption Assistance Using Large Language Models/ Lucas Medeiros Confortin Rosa. – Brasília, DF, 2025-
144 p. : il. (algumas color.) ; 30 cm.

Supervisor: Dr. Fabrício Ataides Braz

Undergraduate Thesis – Universidade de Brasília – UnB
Faculdade UnB Gama – FCTE , 2025.

1. LLM. 2. AWS. I. Dr. Fabrício Ataides Braz. II. Universidade de Brasília. III. Faculdade UnB Gama. IV. AWS Cloud Adoption Assistance Using Large Language Models

CDU 02:141:005.6

Lucas Medeiros Confortin Rosa

AWS Cloud Adoption Assistance Using Large Language Models

Thesis submitted to the Undergraduate Program in Software Engineering at the Universidade de Brasília, as a partial requirement for obtaining the degree of Bachelor in Software Engineering.

Approved thesis. Brasília, DF, February 20, 2025 – Date of thesis approval:

Dr. Fabrício Ataides Braz
Supervisor

Dr. Nilton Correia da Silva
Guest 1

Dr. Henrique Marra Taira Menegaz
Guest 2

Brasília, DF
2025

Dedico este trabalho ao meu filho, Javi, minha principal fonte de motivação diária. Sua alegria, curiosidade e energia me inspiram a ser uma pessoa melhor a cada dia. Tudo o que faço é pensando em você e no futuro que quero construir ao seu lado. Esta conquista é para você, meu pequeno grande motivador. Dedico também ao mais novo membro da família, que ainda não chegou mas já enche a casa de alegria.

Acknowledgements

Agradeço, primeiramente, aos meus amigos da faculdade (ou melhor, inimigos) que fizeram dessa jornada uma experiência muito mais leve e agradável. Vocês foram parte essencial deste caminho, oferecendo apoio, risadas e companheirismo nos momentos mais desafiadores.

Ao meu professor orientador, Dr. Fabrício, deixo meu sincero agradecimento pela orientação paciente e pelo constante auxílio na elaboração deste trabalho. Seu conhecimento e disposição em me guiar foram fundamentais para a concretização deste projeto.

À minha família, meu porto seguro, que sempre me incentivou e acreditou em mim, expresso minha gratidão imensurável. E, em especial, à minha esposa, Jadine, que esteve ao meu lado em todos os momentos difíceis durante a elaboração deste TCC: sua força, compreensão e amor me sustentaram quando pensei em desistir. Sem ela, eu jamais teria conseguido finalizar este documento. Este trabalho é tanto meu quanto dela.

Abstract

The exponential growth of cloud computing over the past decade has transformed IT infrastructure, with services like AWS, Azure, and Google Cloud Platform becoming integral to businesses worldwide. This surge is driven by the scalability, cost-effectiveness, and enhanced security of cloud solutions, as evidenced by a 56% annual growth in cloud service spending between 2009 and 2019. Despite these advantages, the extensive range of services offered by platforms like AWS presents a significant challenge for organizations. Navigating this complex ecosystem requires a deep understanding of numerous services, their interactions, and potential trade-offs, which can be daunting for many users.

To address this challenge, this thesis proposes the development of a cloud adoption assistance system leveraging Large Language Models (LLMs). By utilizing LLMs, the system can generate initial AWS system designs based on high-level service descriptions, subsequently refining these designs through interactive questioning. This approach reduces the expertise and time required to master AWS services, leading to more efficient and optimized cloud architectures. Additionally, the system will translate these designs into Cloud Development Kit (CDK) templates, automating infrastructure provisioning and minimizing human error. This research aims to streamline the cloud adoption process, enabling organizations to build secure, scalable, and cost-effective solutions on AWS.

Key-words: Cloud Computing; AWS; Amazon Web Services; Cloud Adoption; Large Language Models; LLMs; Cloud Infrastructure; Scalability; Cost-Effectiveness; Cloud Development Kit; CDK; AI-Driven Design; Automated Provisioning; IT Infrastructure; Service Optimization; AWS Architecture; AI in Cloud Computing.

List of Figures

Figure 1 – AWS Services Evolution 22
Figure 2 – GPTPilot requesting application description 31
Figure 3 – GPTPilot follow up questions 32
Figure 4 – GPTPilot final requirements 33
Figure 5 – GPTPilot development plan 42
Figure 6 – AWS Chat Application Diagram 45
Figure 7 – HighByte Intelligence Hub Industrial DataOps on AWS 46
Figure 8 – Broadcast-grade live Streaming 47
Figure 9 – Requirements elicitation result 50
Figure 10 – Architecture result 51
Figure 11 – End-to-end use cases result 52
Figure 12 – User registration use case 53
Figure 13 – Search user use case 53
Figure 14 – Start chat use case 54
Figure 15 – Send and receive message use case 54
Figure 16 – Receive notification use case 55
Figure 17 – Overall architecture diagram 55
Figure 18 – Development plan result 56

List of Tables

List of abbreviations and acronyms

IT	Information Technology
AWS	Amazon Web Service
LLM	Large Language Model
CDK	Cloud Development Kit
AI	Artificial Intelligence
NLP	Natural Language Processing
GCP	Google Cloud Platform
S3	Amazon Simple Storage Service
EC2	Amazon Elastic Compute Cloud
RDS	Amazon Relational Database Service
EBS	Amazon Elastic Block Store
GPT	Generative Pre-trained Transformer

Contents

1 INTRODUCTION . . . 21
1.1 Context . . . 21
1.2 Problem . . . 22
1.3 Proposed Solution . . . 23
1.4 Goals . . . 23
1.4.1 General Goal . . . 23
1.4.2 Specific Goals . . . 23

2 THEORETICAL REFERENCE . . . 25
2.1 Large Language Models . . . 25
2.2 Prompt Engineering . . . 25
2.3 GPTPilot . . . 26
2.4 Amazon Web Services . . . 27
2.5 Chat Completions API by OpenAI . . . 27
2.5.1 Basic Usage and Parameters . . . 27
2.5.2 Expected Output . . . 28
2.5.3 Applications and Implications . . . 28

3 MATERIALS AND METHODOLOGY . . . 29
3.1 Initial Considerations . . . 29
3.2 Methodological Plan . . . 29
3.2.1 Select an open source base to the project . . . 29
3.2.2 Adapt requirements elicitation . . . 31
3.2.2.1 GPTPilot requirements elicitation . . . 31
3.2.2.2 Adapted requirement elicitation . . . 33
3.2.3 Adapt the architecture generation . . . 35
3.2.3.1 GPTPilot architecture generation . . . 35
3.2.3.2 Adapted architecture generation . . . 37
3.2.4 Generate end-to-end use cases . . . 40
3.2.5 Generate end-to-end use cases diagrams . . . 40
3.2.6 Generate architecture diagram . . . 41
3.2.7 Adapt development plan generation . . . 42
3.2.7.1 GPTPilot development plan generation . . . 42
3.2.7.2 Adapted development plan generation . . . 43
3.2.7.3 Evaluate generated solution . . . 45
3.2.7.3.1 Live Chat Application . . . 45

3.2.7.3.2 Data Warehouse . . . 46
3.2.7.3.3 Live Streaming Platform . . . 46
3.3 Libraries . . . 47
3.3.1 Jinja2 . . . 47

4 RESULTS . . . 49
4.1 Requirements elicitation . . . 49
4.2 Architecture generation . . . 50
4.3 End-to-end use cases generation . . . 51
4.4 End-to-end use cases diagrams . . . 52
4.5 Architecture diagram . . . 55
4.6 Development plan generation . . . 56
4.7 Evaluate generated solution . . . 56
4.7.1 Live chat application . . . 57
4.7.1.1 Generated architecture . . . 57
4.7.1.2 Comparison . . . 58
4.7.2 Data warehouse . . . 59
4.7.2.1 Generated architecture . . . 59
4.7.2.2 Comparison . . . 61
4.7.3 Live streaming platform . . . 61
4.7.3.1 Generated architecture . . . 61
4.7.3.2 Comparison . . . 63
4.8 Summary of results . . . 63

BIBLIOGRAPHY . . . 65

APPENDIX 67

APPENDIX A – GPTPILOT REQUIREMENTS ELICITATION PROMPT 69

APPENDIX B – CUSTOM REQUIREMENTS ELICITATION PROMPT 75

APPENDIX C – GPTPILOT ARCHITECTURE GENERATION PROMPT 83

APPENDIX D – CUSTOM ARCHITECTURE GENERATION PROMPT 87

APPENDIX E – CUSTOM DEVELOPMENT PLAN GENERATION PROMPT . . . 93

APPENDIX F – REQUIREMENTS ELICITATION RESULT 97

APPENDIX G – ARCHITECTURE RESULT 101

APPENDIX H – END-TO-END USE CASE RESULT 109

APPENDIX I – END-TO-END USE CASE DIAGRAMS RESULT 125

APPENDIX J – OVERALL ARCHITECTURE DIAGRAM RESULT 129

APPENDIX K – DEVELOPMENT PLAN RESULT 133

APPENDIX L – CUSTOM DEVELOPMENT PLAN GENERATION PROMPT 141

1 Introduction

1.1 Context

The past decade has witnessed a phenomenal rise in cloud computing adoption. From a nascent technology in the early 2010s, cloud services have become an indispensable part of modern IT infrastructure. This surge can be attributed to several factors. Firstly, the entry of tech giants such as Amazon, Microsoft, and Google into the cloud arena with robust offerings like AWS, Azure, and Google Cloud Platform (GCP) made these services accessible to a wider audience. Synergy Research Group reports that cloud service spending grew at a staggering 56% annually from 2009 to 2019, dwarfing the 4% average annual growth seen in data center spending during the same period (Synergy Research Group, 2020). Secondly, the scalability and cost-effectiveness of cloud solutions proved highly attractive to businesses of all sizes. Companies no longer needed hefty upfront investments in hardware and software, and could now pay only for the resources they utilized. Finally, advancements in security protocols and compliance certifications addressed initial concerns about data privacy in the cloud, paving the way for wider enterprise adoption.

Amazon Web Services (AWS), a pioneer in the cloud market, exemplifies this growth trajectory. Over the years, AWS has relentlessly expanded its service portfolio, encompassing everything from storage (S3) and compute (EC2) to databases (RDS) and artificial intelligence (SageMaker). This comprehensive suite caters to diverse business needs and fosters a dynamic cloud ecosystem.

Figure 1 – AWS Services Evolution



Source: (XU, 2023)

1.2 Problem

However, the very features that fuel AWS's dominance – it's expansive service portfolio and unmatched scalability – also introduce a new hurdle for cloud adopters. While the initial surge in cloud computing was driven by the convenience and cost-effectiveness of these services, as exemplified by AWS's growth, navigating the intricacies of this vast ecosystem presents a significant learning curve. Unlike traditional on-premise IT environments with a limited set of tools, AWS offers hundreds of services, each with its own functionalities, pricing models, and use cases. Mastering this complex landscape requires a deep understanding of individual services, their potential trade-offs, and the intricate ways they can be interconnected. Building a truly scalable, reliable, and efficient system on AWS demands the ability to orchestrate these services in a cohesive manner.

For instance, choosing between Amazon S3 and EBS for storage depends on factors like access frequency, data persistence needs, and cost optimization. Similarly, integrating services like an Amazon Relational Database Service (RDS) instance with an Auto Scaling group for web applications necessitates an understanding of their interaction patterns and potential bottlenecks. This knowledge gap can be a hurdle for organizations transitioning to the cloud, potentially leading to suboptimal deployments or unforeseen costs.

1.3 Proposed Solution

One potential solution to bridge the knowledge gap and streamline AWS adoption is the utilization of Large Language Models (LLMs). These AI models, trained on vast amounts of data, can be leveraged to generate initial AWS system design alternatives based on a high-level service description. The LLM would analyze the use case, identify relevant AWS services, and propose a preliminary architecture. This initial design could then be further refined through a series of follow-up questions posed to the LLM. These questions could delve into specific requirements like scalability needs, cost constraints, and security considerations. By interactively refining the service selection and configuration based on the LLM's responses, users can arrive at a more optimized AWS architecture. This LLM-driven approach could significantly reduce the time and expertise needed to navigate the complex AWS service landscape.

Furthermore, the generated design could be directly translated into a Cloud Development Kit (CDK) template by another LLM specializing in code generation. This would automate the infrastructure provisioning process, ensuring consistency and reducing the risk of human error. By leveraging the combined power of these LLMs, organizations can streamline their cloud adoption journey and build secure, scalable, and cost-effective AWS solutions.

1.4 Goals

1.4.1 General Goal

To develop a cloud adoption assistance system that leverages Large Language Models to automate AWS system design and infrastructure provisioning based on high-level service description

1.4.2 Specific Goals

1. Define the project requirements from a brief high-level description and some follow-up questions.

- Modify GPTPilot to refine the requirements and create artifacts, such as user stories and use cases, using ChatGPT.

2. Use a Large Language Model conversational agent to create a low level AWS architecture design using the requirements as input.
   - Modify GPTPilot to create the AWS architecture using ChatGPT.
   - Modify GPTPilot to, based on the architecture, generate a description of how use cases interact with AWS resources.
3. Use a large language model to create diagrams using the low-level architecture and use cases as input.
   - Modify GPTPilot to generate a mermaid graph for each use case that illustrates it.
   - Modify the project to generate a mermaid graph description that can be used to generate an architecture diagram.
4. Use a large language model to generate a development plan to implement the architecture.
   - Modify GPTPilot to use the architecture and project description to generate a development plan.
5. Validate that the system is generating a cohesive AWS architecture.
   - Search for established design standards on the AWS architecture.
   - Generate an AWS architecture.
   - Compare the market standard with the system's output.

2 Theoretical Reference

2.1 Large Language Models

Large Language Models are a subset of artificial intelligence and natural language processing technologies designed to understand, generate, and manipulate human language. These models are typically built using deep learning techniques, specifically neural networks with many layers, which enable them to learn from vast amounts of text data and perform a wide range of language-related tasks (BROWN et al., 2020).

One of the key characteristics of LLMs is their scale. They are trained on extensive datasets that often include billions of words from diverse sources such as books, articles, websites, and social media. The training process involves adjusting the weights of the neural network based on the input data, enabling the model to recognize patterns and relationships within the text. The size of these models, measured in terms of parameters (weights), can reach hundreds of billions, making them capable of capturing nuanced language patterns and context (DEVLIN et al., 2019).

The applications of LLMs are broad and impactful. They are used in virtual assistants, customer service bots, content creation tools, and more. In the research community, they aid in exploring new frontiers in understanding human language and developing innovative language-based technologies. However, the deployment of LLMs also raises ethical considerations, such as the potential for biased or harmful outputs, the environmental impact of training large models, and the need for responsible use and regulation (BENDER et al., 2021).

In conclusion, large language models represent a significant leap forward in the field of AI and NLP. Their ability to understand and generate human language with high accuracy and context sensitivity opens up numerous possibilities for advancements in technology and industry. As these models continue to evolve, ongoing research and ethical considerations will be crucial in harnessing their full potential while mitigating associated risks.

2.2 Prompt Engineering

Prompt engineering is a critical discipline within artificial intelligence that involves crafting input prompts to effectively direct large language models toward generating desired outputs. By meticulously designing these prompts, users can significantly influence

the performance of LLMs, enhancing their ability to produce accurate and contextually appropriate responses (LIU et al.,).

Unlike traditional model training methods, prompt engineering leverages the pre-existing knowledge of LLMs without the need for additional fine-tuning. Techniques such as few-shot prompting provide the model with a handful of examples within the prompt, enabling it to perform specific tasks more effectively (Brown et al., 2020). Other methods like prompt tuning involve adjusting continuous prompt embeddings to guide the model's behavior (LI; LIANG,).

The development of prompt engineering has opened up new avenues in various applications, including text summarization, sentiment analysis, and code generation. It serves as a bridge between human intent and machine understanding, allowing for more intuitive interactions with AI systems. Researchers are also exploring automated prompt generation to reduce reliance on manual prompt design, employing algorithms to create prompts that elicit optimal responses from LLMs (GAO; FISCH; CHEN, 2020).

2.3 GPTPilot

> GPT Pilot is the core technology for the Pythagora VS Code extension that aims to provide the first real AI developer companion. Not just an autocomplete or a helper for PR messages but rather a real AI developer that can write full features, debug them, talk to you about issues, ask for review, etc.(PYTHAGORA-IO, 2024)

This open-source tool designed to generate up to 95% of the code required to build an application, significantly enhancing the efficiency of the development process. The project, available on GitHub, exemplifies an innovative approach to AI-driven development. GPTPilot not only assists with basic coding tasks but also takes on complex roles traditionally performed by human developers, such as debugging, reviewing, and technical writing.

How GPTPilot works:

1. You enter the app name and the description.
2. The Product Owner agent, mimicking real-life dynamics, remains passive.
3. The Specification Writer agent asks questions to clarify requirements if the project description is insufficient.
4. The Architect agent outlines the technologies to be used for the app, checks their availability on the machine, and installs them if necessary.
5. The Tech Lead agent drafts development tasks that the Developer agent must implement.
6. The Developer agent details the steps needed to complete each task in a human-readable format.
7. The Code Monkey agent takes the Developer's instructions and implements the necessary changes in the code.

8. The Reviewer agent evaluates each task and sends it back to the Code Monkey agent if revisions are needed.
9. The Troubleshooter agent assists in providing constructive feedback to GPTPilot when issues arise.
10. The Debugger agent, although rarely desired, is invaluable when problems need resolution.
11. The Technical Writer agent generates documentation for the project.(PYTHAGORA-IO, 2024)

2.4 Amazon Web Services

Amazon Web Services is a comprehensive and widely adopted cloud computing platform provided by Amazon.com Inc. It offers a diverse range of cloud services including computing power, storage options, networking, databases, machine learning, and analytics, among others. AWS enables businesses and developers to access scalable and flexible IT infrastructure resources on a pay-as-you-go basis, without the need for substantial upfront investment in hardware and infrastructure management.(WHAT..., 2024)

AWS was launched in 2006 with the goal of providing businesses with access to the same scalable and reliable technology infrastructure that Amazon.com used to handle its vast online retail operations. Over the years, AWS has grown to become one of the leading cloud service providers globally, serving millions of customers ranging from startups to large enterprises and government agencies.

Key features of AWS include its global infrastructure footprint, which spans multiple geographic regions and availability zones, ensuring high availability and fault tolerance for applications and services. AWS's service portfolio covers a wide spectrum of cloud computing needs, from basic compute instances (EC2) to advanced machine learning services (Amazon SageMaker) and serverless computing (AWS Lambda).

2.5 Chat Completions API by OpenAI

This API leverages models developed by OpenAI, such as the GPT (Generative Pre-trained Transformer) series, to generate human-like text responses based on the input provided by the user (OPENAI, 2021).

2.5.1 Basic Usage and Parameters

The API is designed to be straightforward to use, with several key parameters that control its behavior:

- **model**: Specifies the particular model version to be used, e.g., *gpt-4* or *gpt-3.5-turbo*.

- **messages**: An array of message objects representing the conversation history, where each message includes a *role* (such as 'system', 'user', or 'assistant') and *content*.
- **max_tokens**: Defines the maximum number of tokens (words and pieces of words) that the response can contain.
- **temperature**: Controls the randomness of the response, with higher values leading to more varied outputs.
- **top_p**: Determines how much the API focuses on high probability completions, shaping the response's predictability.

2.5.2 Expected Output

Upon a successful API call, the response includes a *choices* array containing objects with *text* entries, which are the generated completions of the input prompt. These completions can be utilized in various applications such as chatbots, virtual assistants, and automated customer support systems, showcasing the API's flexibility and utility in real-world scenarios.

2.5.3 Applications and Implications

The API's potential applications are vast, from enhancing conversational interfaces to creating dynamic narrative experiences. Its ability to understand context and generate coherent, contextually appropriate responses makes it an invaluable tool in the development of AI-driven interaction systems.

3 Materials and Methodology

3.1 Initial Considerations

In this chapter, the materials and methods used to achieve the objectives of this thesis are presented. The methodological plan adopted will be detailed, outlining the steps that will be followed to adapt GPTPilot to convert high-level specifications of an application into a functional AWS architecture.

For the purposes of this section, all examples of GPTPilot utilization will be demonstrated using OpenAI's Chat Completions API with the **o1-mini** model. The following application description will be used as a base:

I want to create a chat app. An user must be able to login with an username, search other user by his username and start a chat.

3.2 Methodological Plan

The methodological plan has eight steps:

- Research and select an open source *specification to application* software to use as base to this project;
- Adapt the requirements elicitation stage for the purposes of this thesis;
- Adapt the architecture generation stage for the purposes of this thesis;
- Generate the end-to-end use cases based on the architecture generated;
- Generate a mermaid diagram to illustrate the end-to-end use cases description;
- Generate a mermaid diagram to illustrate the architecture description;
- Adapt the development plan generation stage for the purposes of this thesis;
- Evaluate if the generated solution aligns with standards and best practices of AWS architecture.

3.2.1 Select an open source base to the project

To develop a system capable of generating AWS architectures based on user requirements using large language models, it was essential to choose an appropriate open-source foundation that could be adapted and extended to meet the project's objectives.

After evaluating some AI-powered code generation projects, **GPTPilot** was selected as the base for this project.

GPTPilot is an open-source application that leverages OpenAI's GPT models to automate software development tasks. It provides a framework for interacting with users to gather project requirements and generate code based on those specifications. GPTPilot's modular design and extensibility make it suitable for customization and adaptation to different domains, such as AWS architecture generation.

Several factors influenced the decision to choose GPTPilot over other AI-powered code projects:

- **Open-Source Availability:** Unlike some proprietary solutions, GPTPilot is open-source, allowing full access to its codebase for modification and extension without licensing restrictions. This openness is crucial for academic projects that require transparency and the ability to customize underlying mechanisms.
- **Focus on Requirement Elicitation:** GPTPilot emphasizes the process of gathering detailed requirements from users through conversational interactions. This focus aligns with the project's goal of refining user inputs to generate accurate AWS architectures, making it a suitable foundation.
- **Modular and Extensible Architecture:** GPTPilot's design allows for easy integration of custom prompts and the modification of its conversational interface. This flexibility is essential for incorporating prompt engineering techniques aimed at improving output quality.

In comparison to other available AI-powered code projects:

- **OpenAI Codex and GitHub Copilot:** While powerful in code generation and assistance, these tools are not open-source and have usage limitations that restrict their adaptability for specific project needs. They are also designed primarily for code completion and may not offer the same level of customization in requirement elicitation.
- **TabNine and Kite:** These are AI-powered code completion tools integrated into code editors. They assist developers by predicting and suggesting code snippets but do not focus on generating entire project architectures based on user requirements.
- **Polyglot Coding Assistants:** Some projects offer multilingual code assistance but lack the capability to engage in detailed requirement elicitation and generate comprehensive architectural designs.

By selecting GPTPilot as the project's base, we leverage an open-source platform that closely aligns with our objectives. Its existing framework for requirement gathering and code generation provides a solid foundation upon which we can implement custom prompt engineering techniques and adapt the system to generate AWS architectures effectively. This choice enables us to focus on enhancing the system's capabilities without the constraints imposed by proprietary software, ensuring that the project's outcomes are both innovative and accessible for future research and development.

3.2.2 Adapt requirements elicitation

3.2.2.1 GPTPilot requirements elicitation

The requirements elicitation process begins with asking the user to provide a detailed description of the application they want to build.

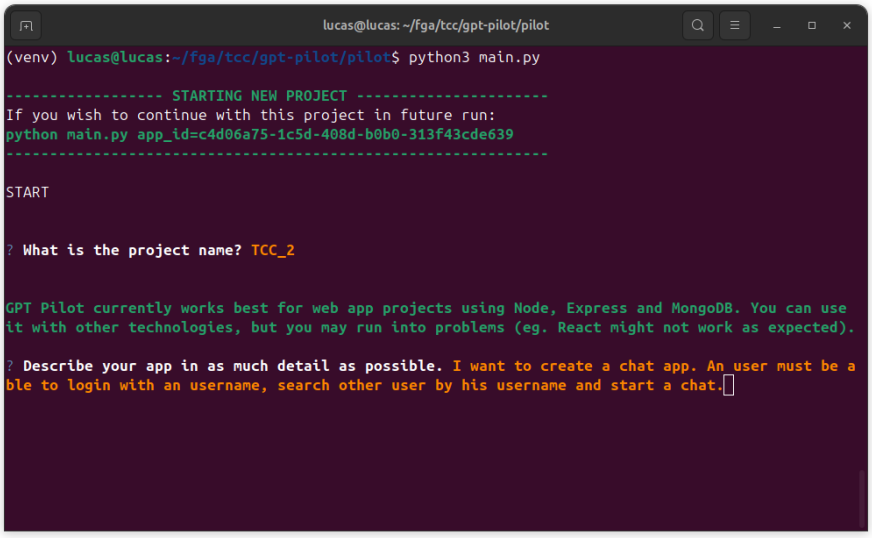


Figure 2 – GPTPilot requesting application description

Using the initial input from the user and a detailed prompt to guide how ChatGPT should handle this input and what the expected outputs are, GPTPilot constructs a conversation object and initiates the first interaction with ChatGPT (the prompt used to generate the requirements can be found on appendix A).

At this stage, depending on the user's input, ChatGPT might ask follow-up questions to refine the project description. These questions could address unclear features, the number of expected users, scalability, security concerns, etc.

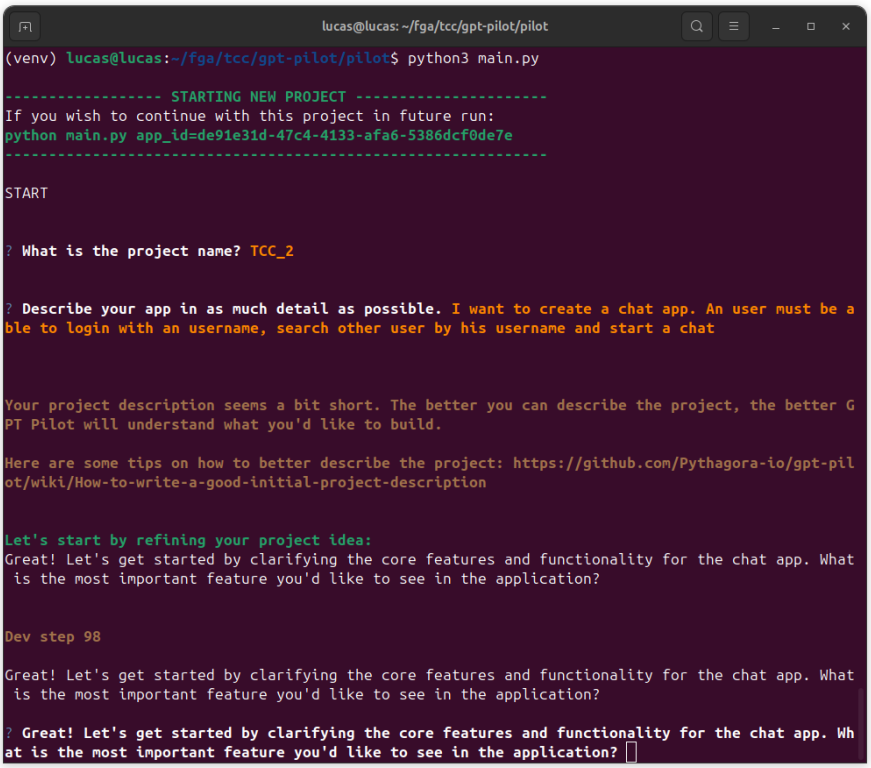


Figure 3 – GPTPilot follow up questions

Once all follow-up questions have been addressed, or if the initial description is deemed sufficient, ChatGPT will generate a complete list of project requirements. At this point, the user can request changes to the requirements. If changes are requested, the previous steps will be repeated, and a new list of requirements will be produced.

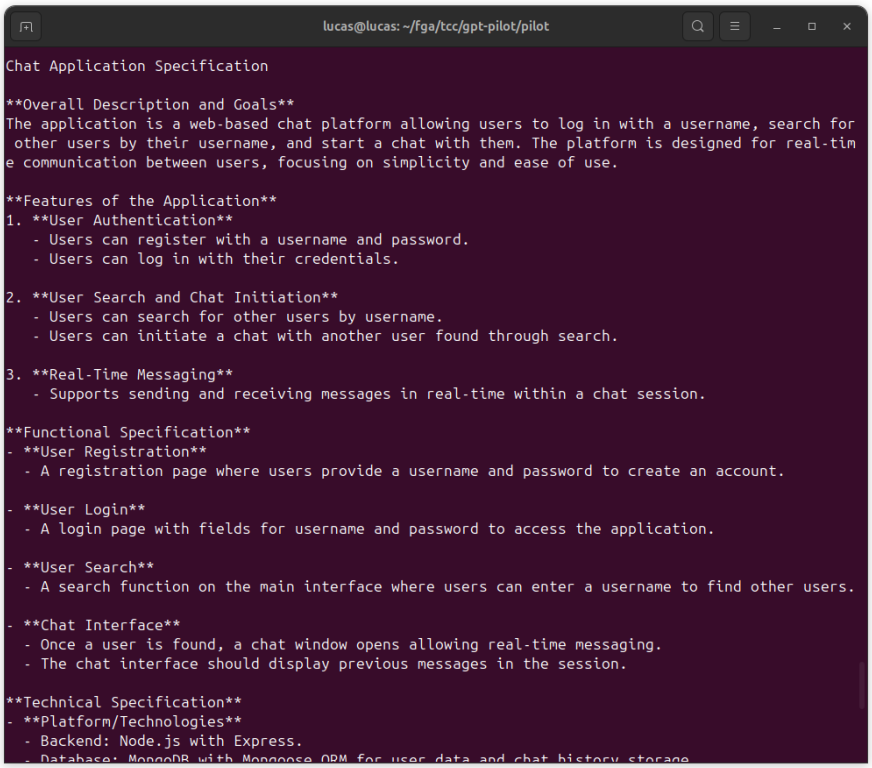


Figure 4 – GPTPilot final requirements

3.2.2.2 Adapted requirement elicitation

The overall requirements elicitation workflow remained the same. The user provide a description to the project, the assistant refine the idea and generates a requirements list. The main change was to enhance the requirements refinement process by improving the prompt used to this task. These modifications aim to produce more accurate, user-centered, and functionally detailed requirements while avoiding unnecessary technical jargon or overwhelming the user with excessive questioning. Below, we highlight the specific changes made to the prompt to accomplish each of these improvements. The full prompt is available on appendix B.

1. Emphasizing Functional Requirements Over Technical Details

Changes Made: The original prompt guided the assistant to include both functional and technical specifications. To shift the focus solely to functional requirements, the adapted prompt explicitly states:

> *"...must accurately capture all the functional aspects and user needs without addressing technical implementation details."*

This change directs the assistant to concentrate on what the application should do from the user's perspective, rather than how it should be built.

2. Structured and Organized Output Guidance

Changes Made: To ensure the assistant provides a well-organized specification, instructions were added:

> *"Provide a complete requirements specification that includes all gathered information. Use clear headings, subheadings, and bullet points for readability."*

This modification encourages a systematic structure, enhancing readability and making it easier for stakeholders to navigate the document.

3. Limited and Prioritized Questioning

Changes Made: The original prompt allowed unlimited questions, which could overwhelm the client. The adapted prompt limits this by stating:

> *"You may ask up to **7 follow-up questions**; prioritize those that gather essential information."*

By capping the number of questions, the assistant is prompted to prioritize and ask only the most critical ones.

4. User-Centric Focus

Changes Made: To place greater emphasis on user experience, the prompt now includes:

> *"Understand how the client envisions the user interface and interactions."*
>
> *"Inquire about any specific user roles and permissions."*

These additions ensure the assistant gathers detailed information about user interactions and roles.

5. Avoidance of Trivial Questions

Changes Made: To prevent unnecessary queries, the prompt advises:

> *"Avoid trivial questions about minor details unless they significantly impact functionality."*

This change helps maintain the client's engagement by focusing on significant aspects.

6. Inclusion of Examples for Clarity

Changes Made: An illustrative example was added:

"Example of Transforming a User Description into Requirements:"

[Example follows in the prompt.]

Providing an example helps the assistant understand the expected output format and level of detail, facilitating better compliance with the desired outcome.

7. Iterative Updates for Completeness

Changes Made: To ensure the specification remains current, the prompt adds:

"If the client provides new information or changes, update the entire requirements specification accordingly."

This encourages the assistant to keep the document up-to-date, reflecting any modifications.

3.2.3 Adapt the architecture generation

3.2.3.1 GPTPilot architecture generation

In the architecture generation phase, GPTPilot leverages the list of requirements obtained from the requirements elicitation step to produce a detailed system architecture. Unlike the previous phase, this step operates without any additional user interaction. The assistant combines the requirements list with a predefined prompt ,provided in the appendix C, to instruct it to generate the architecture in a specific JSON format. This format encompasses key components such as the overall architecture description, system dependencies, package dependencies, and the project template to be used. The general structure of the expected JSON output is as follows:

```
{
    "architecture": "Detailed description of the architecture of the application",
    "system_dependencies": [
        {
            "name": "Node.js",
            "description": "JavaScript runtime for building apps. This is required to
            be able to run the app you're building.",
            "test": "node --version",
            "required_locally": true
        },
        {
            "name": "MongoDB",
            "description": "NoSQL database. If you don't want to install MongoDB
            locally, you can use a cloud version such as MongoDB Atlas.",
            "test": "mongosh --version",
```

```
            "required_locally": false
        },
        ...
    ],
    "package_dependencies": [
        {
            "name": "express",
            "description": "Express web server for Node"
        },
        ...
    ],
    "template": "name of the project template to use"
}
```

For the example provided in Section 3.2.2.1, the assistant generates the following architecture output:

```
{
    "architecture": "The application utilizes a client-server architecture with
    Node.js and Express on the backend, MongoDB as the database, and EJS with
    Bootstrap on the frontend. Socket.io is used for real-time messaging.",
    "system_dependencies": [
        {
            "name": "Node.js",
            "description": "JavaScript runtime for building apps. This is required to
            be able to run the app you're building.",
            "test": "node --version",
            "required_locally": true
        },
        {
            "name": "MongoDB",
            "description": "NoSQL database. If you don't want to install MongoDB
            locally, you can use a cloud version such as MongoDB Atlas.",
            "test": "mongosh --version",
            "required_locally": false
        }
    ],
    "package_dependencies": [
        {
            "name": "express",
            "description": "Express web server for Node"
        },
        {
            "name": "mongoose",
            "description": "MongoDB object modeling tool for Node.js"
        },
        {
```

```
            "name": "ejs",
            "description": "Embedded JavaScript templating"
        },
        {
            "name": "bcrypt",
            "description": "Library to hash passwords"
        },
        {
            "name": "express-session",
            "description": "HTTP sessions middleware for Express"
        },
        {
            "name": "socket.io",
            "description": "Enables real-time bidirectional event-based communication"
        },
        {
            "name": "dotenv",
            "description": "Loads environment variables from a .env file"
        }
    ],
    "template": "node_express_mongoose"
}
```


This JSON output provides a comprehensive overview of the system architecture, including the technologies to be used, system dependencies with installation tests, necessary package dependencies, and the project template. By automating this step, GPTPilot streamlines the development process, ensuring consistency between the requirements and the proposed architecture without additional input from the user.

3.2.3.2 Adapted architecture generation

To improve the architecture design process and enhance the quality of the generated system architecture, several prompt engineering techniques were applied to modify the original GPTPilot prompt, appendix D. These modifications aim to produce a more AWS-focused, structured, and comprehensive architectural specification that aligns closely with the project requirements. Below, we highlight the specific changes made to the prompt to accomplish each improvement.

1. Emphasis on AWS Services and AWS CDK with Python

Changes Made: The original prompt was technology-agnostic, defaulting to Node.js, Express, and MongoDB if no specific technologies were mentioned. To focus on AWS services, the adapted prompt explicitly states:

"You are designing the architecture and technical specifications for a new project that will be entirely hosted on AWS, utilizing appropriate AWS services. The architecture will be implemented using AWS CDK with Python."

This change directs the assistant to design architectures that leverage AWS offerings, ensuring the solution is cloud-native and aligns with industry best practices for cloud deployments.

2. Logical Grouping into CDK Stacks

Changes Made: The original prompt did not specify how to structure the architecture components. The adapted prompt instructs:

"Your goal is to create an architecture composed of CDK stacks, where each stack represents a logical grouping of related application components (e.g., networking, databases, authentication), rather than being based on specific use cases."

This encourages modular design, promoting separation of concerns and easier maintenance.

3. Structured and Detailed Instructions

Changes Made: The adapted prompt provides a clear, step-by-step structure for the assistant to follow:

"1. High-Level Architecture Overview... 2. Detailed CDK Stack Descriptions... 3. Assumptions and Considerations..."

This contrasts with the original prompt, which had a less detailed structure. The explicit instructions help ensure that all necessary aspects are covered systematically.

4. Inclusion of Interactions and Justifications

Changes Made: To deepen the understanding of the architecture, the adapted prompt adds:

"Describe how the stacks will interact with each other."

"Explain why this architecture is the best option for this project."

By requesting interactions and justifications, the assistant is prompted to think critically about architectural decisions.

5. Formatting Guidelines for Clarity

Changes Made: The adapted prompt includes specific formatting instructions:

"Use clear headings and subheadings to organize your output. Present information in bullet points or numbered lists where appropriate."

This enhances the readability of the output.

6. Provision of an Example Output

Changes Made: An example is provided to illustrate the expected format and level of detail:

*"**Example:**... [Detailed example follows in the prompt.]"*

Including an example helps set clear expectations and guides the assistant in producing a suitable response, a recognized technique in prompt design called few-shot learning (GAO; FISCH; CHEN, 2020).

7. Avoidance of Unnecessary Technical Details

Changes Made: The adapted prompt advises:

"Avoid technical implementation details; focus on architectural design."

This ensures the assistant remains focused on high-level architectural considerations rather than delving into low-level implementation specifics.

8. Focus on Assumptions and Considerations

Changes Made: The adapted prompt explicitly requests:

"Note any assumptions you've made in your architectural design. Mention any constraints that influenced your design decisions."

By identifying assumptions and constraints, the assistant provides context for design decisions.

9. Emphasis on Scalability and Security

Changes Made: The prompt includes:

"Briefly address how the architecture accommodates scalability and security needs."

This addition aims that the architecture considers non-functional requirements.

10. Removal of Irrelevant Instructions

Changes Made: The original prompt included instructions about system dependencies, package dependencies, and project templates, which are less relevant in the context of AWS architecture design using CDK. The adapted prompt removes these sections to keep the focus on designing the AWS architecture.

11. Tailoring to AWS Services and Best Practices

Changes Made: The assistant is instructed to select AWS services that best meet the project requirements:

> *"...design the architecture step by step, choosing AWS services that best meet the project requirements."*

This change aligns the assistant's response with AWS best practices and service offerings, ensuring that the architecture is optimized for the AWS environment.

The full prompt can be found in appendix D.

3.2.4 Generate end-to-end use cases

Building on the context from previous interactions with the LLM agent, this step focuses on generating comprehensive end-to-end descriptions of the use cases. These descriptions provide a clearer understanding of how the LLM agent intends to utilize various AWS resources and how these services interact to meet specific requirements.

To achieve this, the following prompt was appended to the conversation from the previous step (*Generate Architecture*), ensuring the agent has access to all previously gathered information, such as the user's project description, requirements elicitation, and overall architecture:

```
With the application description and this architecture, describe how the
main use cases will interact with these AWS service.
Describe how the data will be handled and what will be the next step of
an end-to-end workflow
```

3.2.5 Generate end-to-end use cases diagrams

With both the architecture and use case specifications defined, the next step is to enhance the visualization of these artifacts. To achieve this, we will prompt the LLM agent to generate a separate diagram for each use case described in the previous step. These diagrams will be output as Mermaid syntax and subsequently converted into images.

The following prompt was used, appended to previous messages to ensure the agent retains access to the execution context:

```
Create me mermaid diagrams depicting the use cases. Include all the
details of each use case.
You must create one diagram to each use case. Output only the diagrams
with the use case title.
Your diagrams must contain the AWS services used to each use case.
```

3.2.6 Generate architecture diagram

To visualize the AWS architecture designed for the project, another mermaid diagram is utilized to represent the various components, their interactions, and the overall system structure. After generating the detailed architecture description, a carefully crafted prompt is used to instruct the AI model to create a Mermaid diagram that accurately reflects the architecture.

The prompt used to generate the Mermaid diagram is as follows:

```
Using the architecture description provided below, create a detailed
**Mermaid diagram** that visualizes the entire AWS architecture. The
diagram should include:

- **All CDK Stacks**: Networking Stack, Authentication Stack, Application
Stack, Database Stack, Storage Stack, Notification Stack, Search Stack,
and Content Delivery Stack.
- **AWS Resources within each Stack**: Clearly represent all AWS services
used in each stack, such as VPCs, Lambda functions, API Gateway,
DynamoDB, S3 buckets, etc.
- **Interactions and Data Flow**: Illustrate how the stacks and their
components interact with each other, including data flow and
communication paths.
- **User Interactions**: Show how users interact with the system,
including authentication and content delivery.
- **Security Components**: Include security groups, IAM roles, and any
security considerations mentioned.
- **Optional Components**: Represent any optional services or components
specified in the description.
- **Labels and Icons**: Use appropriate labels for clarity. If possible,
include AWS service icons or standard symbols to represent different
services.

[Detailed requirements description]
```

3.2.7 Adapt development plan generation

3.2.7.1 GPTPilot development plan generation

During GPTPilot's development plan generation phase, the project requirements and architecture serve as the foundation for breaking down the project into smaller tasks, which are then assigned to the code agent.

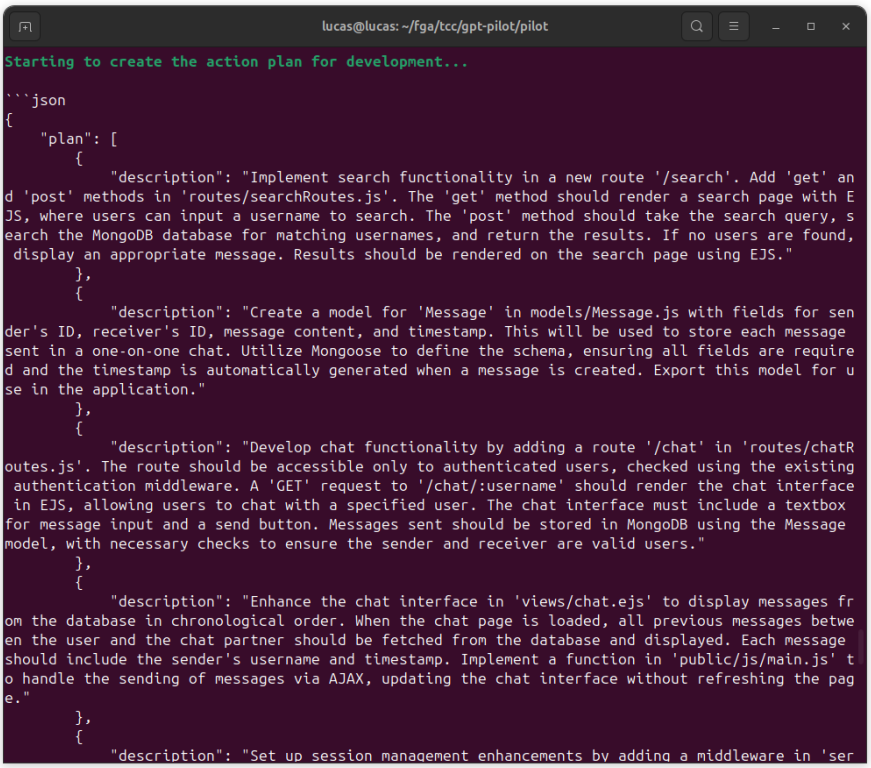


Figure 5 – GPTPilot development plan

The following prompt is used to generate this task breakdown:

```
You are working in a software development agency and a project manager
and software architect approach you telling you that you're assigned to
work on a new project. You are working on a {{ app_type }} called "{{
name }}" and you need to create a detailed development plan so that
developers can start developing the app.

{{ project_details }}
{{ features_list }}
{% if files %}The developers have already used a project scaffolding tool
that creates the initial boilerplate for the project:
{{ existing_summary }}
```

```
{{ files_list }}{% endif %}

{{ project_tasks }}
```

3.2.7.2 Adapted development plan generation

In order to produce a more focused, AWS-centric, and systematically structured development plan, several prompt engineering adjustments were made to the original prompt. This section details the specific changes implemented, demonstrating how the modified prompt guides developers toward a clearer, more maintainable, and code-oriented set of tasks.

1. Emphasis on AWS and AWS CDK with Python

Changes Made: The original prompt generically asked for a "detailed development plan" without specifying a particular platform. In the modified prompt, AWS Cloud services and AWS CDK in Python are explicitly mandated:

> *"You are working on a fully AWS system that will be implemented using AWS CDK with python... You will receive the project main description, the project specification and the selected architecture..."*

This modification is designed to ensure that the development plan leverages AWS best practices and uses a specific infrastructure-as-code framework, thereby fostering a consistent, cloud-native approach.

2. Clear Division by CDK Stacks

Changes Made: In the original prompt, there was no requirement to group or organize tasks by architectural layers. The adapted prompt explicitly requires each AWS CDK stack to be used as the primary grouping mechanism for tasks:

> *"The architecture is divided into CDK stacks... it is the granularity you must use to create the development plan."*

Focusing on stack-based segmentation is meant to promote a plan that emphasizes modular, isolated components, which may simplify future development, maintenance, and debugging of the AWS infrastructure.

3. Strict Focus on Code-Oriented Tasks

Changes Made: The modified prompt explicitly forbids tasks that do not involve coding, such as research, documentation, or high-level testing:

> *"Each task needs to be related only to the development of this application and nothing else... There shouldn't be a task for researching, deployment, writing documentation, testing..."*

Whereas the original prompt allowed any project-related tasks (including documentation, QA, etc.), the new version seeks to ensure that every item in the plan contributes directly to the codebase.

4. Medium Complexity for Each Task

Changes Made: The adapted prompt contains explicit guidelines on task size and complexity:

> *"You must ensure the project is not divided into tasks that are too small or simple... nor too large or complex... Aim for medium complexity tasks."*

Compared to the original prompt, which provided no specific guidance about task scope, the new version strives to promote tasks that are granular enough to be manageable yet substantial enough to represent meaningful progress.

6. Detailed and Self-Contained Task Descriptions

Changes Made: In the original prompt, tasks could be described briefly (e.g., "Create a detailed development plan for the app"). The revised prompt insists each task must include at least four sentences of explanation, specifying all relevant coding details:

> *"Every task must have a clear and very detailed description (minimum of 4 sentences)... Provide as many details as possible..."*

This revision is crafted to eliminate ambiguity and ensure that any developer, even one new to the project, can execute the tasks without additional clarification.

7. Integration of Examples and Counterexamples

Changes Made: The modified prompt provides concrete examples of "good" versus "bad" tasks:

> *"Here is an example of good and bad task descriptions... 'Create a CDK stack for an S3 bucket' (bad) vs. 'Define an S3 bucket with versioning enabled...' (good)."*

Including these examples is aimed at reinforcing the expected level of detail in task descriptions, thereby clarifying the necessary technical information.

The complete development plan prompt can be found on appendix L

3.2.7.3 Evaluate generated solution

To assess the performance of our agent, three systems of increasing complexity were selected:

- Live Chat Application – A platform where users can initiate conversations with others by searching for their username.
- Data Warehouse – A system capable of gathering data from multiple sources and making it available for querying and analysis.
- Live Streaming Platform – A service where content creators can start a live stream, and viewers can join in real-time.

For each of these cases, Amazon provides official guidelines. While is not expect the agent's output to precisely match these official examples, we can evaluate its effectiveness by comparing the AWS resources utilized and the overall workflow of the generated solutions.

3.2.7.3.1 Live Chat Application

For the live chat application, we base our evaluation on *Tutorial: Create a WebSocket Chat App with a WebSocket API, Lambda, and DynamoDB* (Amazon, a). The tutorial provides a simple architectural diagram:

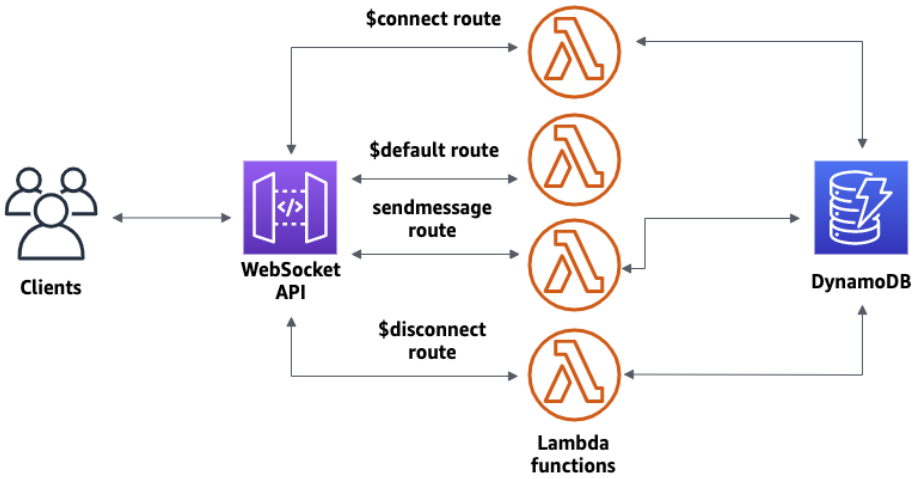


Figure 6 – AWS Chat Application Diagram (Amazon, a)

Therefore, we expect the generated solution to incorporate WebSockets, AWS Lambda functions, and Amazon DynamoDB as core components.

3.2.7.3.2 Data Warehouse

This application is base on AWS Reference Architecture *Integrate Industrial Data with Enterprise Systems in AWS Cloud* (Amazon, 2024):

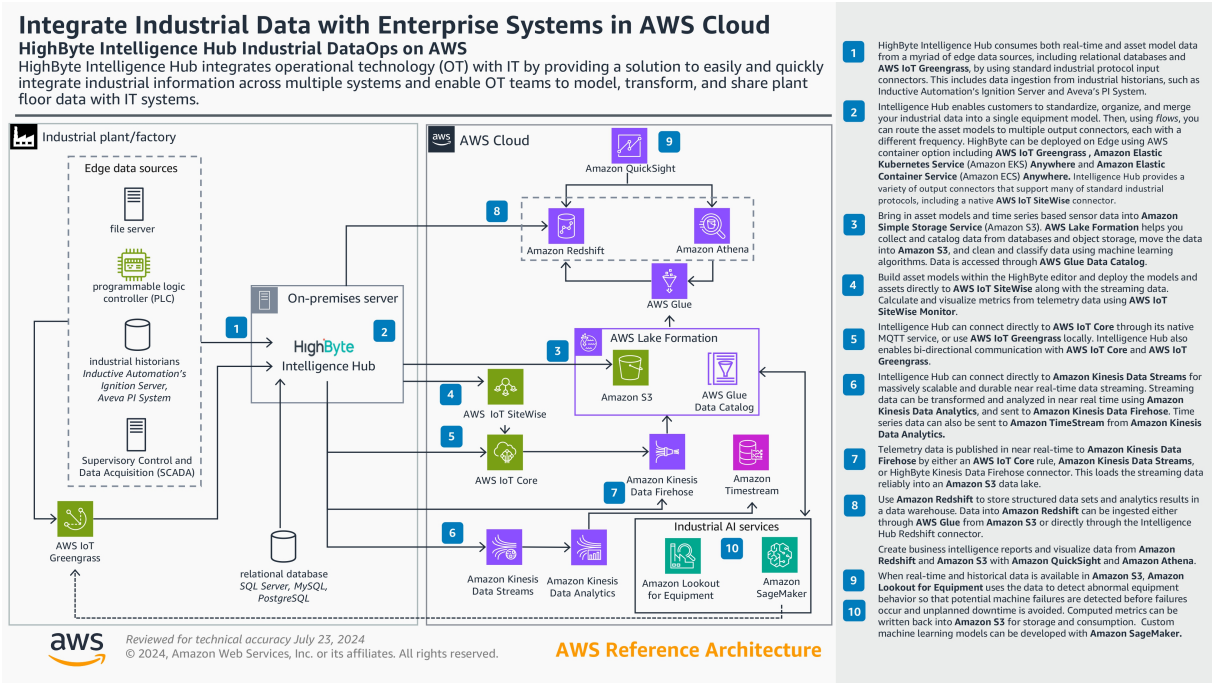


Figure 7 – HighByte Intelligence Hub Industrial DataOps on AWS (Amazon, 2024)

Based on this architecture, we expect the solution to utilize the following AWS services:

- Amazon Kinesis Data Streams Firehose for real-time data streaming;
- Amazon S3 as the primary data lake;
- AWS Glue and AWS Lake Formation for data cataloging and transformation;
- Amazon Redshift for structured data warehousing;
- Amazon Athena for ad-hoc querying;
- Amazon QuickSight for data visualization and reporting.

3.2.7.3.3 Live Streaming Platform

The live streaming solution is evaluated against Video streaming architectures (Amazon, b):

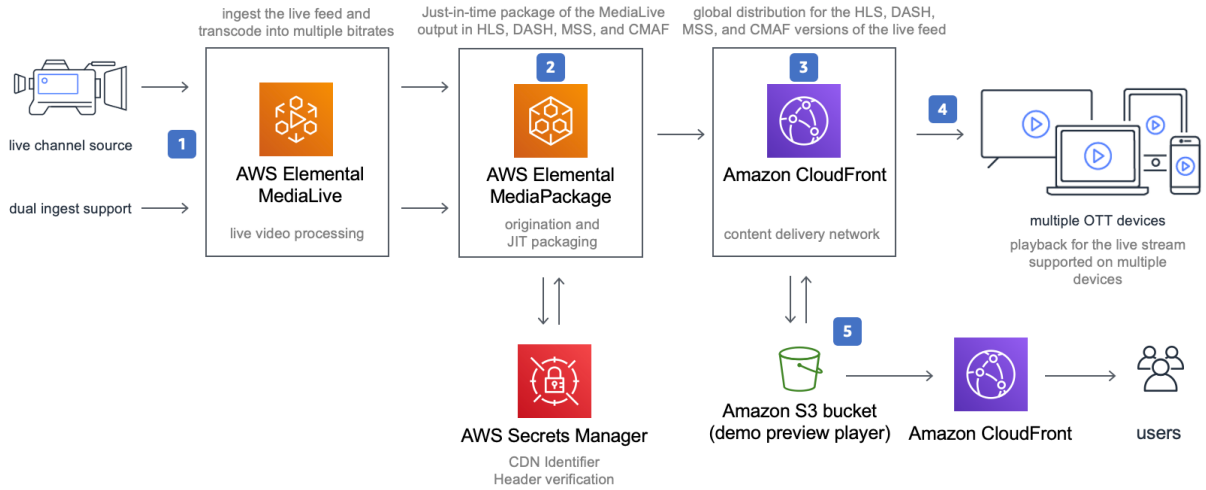


Figure 8 – Broadcast-grade live Streaming
(Amazon, b)

Based on this reference, we anticipate the following AWS services in the generated solution:

- MediaLive for ingesting the live signal;
- MediaPackage ingests the MediaLive ABR output and convert it to multiple formats;
- CloudFront for content delivery;
- Amazon S3 for content storage.

Additionally, although not explicitly shown in the diagram, the reference also mentions Amazon IVS, a fully managed live streaming solution suitable for scenarios with fewer than 15,000 simultaneous connections.

3.3 Libraries

3.3.1 Jinja2

Jinja is a fast, expressive, extensible templating engine. Special placeholders in the template allow writing code similar to Python syntax. Then the template is passed data to render the final document.

It includes:

- Template inheritance and inclusion;
- Define and import macros within templates;

- HTML templates can use autoescaping to prevent XSS from untrusted user input;
- A sandboxed environment can safely render untrusted templates;
- AsyncIO support for generating templates and calling async functions;
- I18N support with Babel;
- Templates are compiled to optimized Python code just-in-time and cached, or can be compiled ahead-of-time;
- Exceptions point to the correct line in templates to make debugging easier;
- Extensible filters, tests, functions, and even syntax.

Jinja's philosophy is that while application logic belongs in Python if possible, it shouldn't make the template designer's job difficult by restricting functionality too much. (JINJA2. . . ,)

4 Results

To evaluate the system's performance, three different requirement inputs were used:

> *"I want to create a chat app. An user must be able to login with an username, search other user by his username and start a chat."*

> *"I want you to create a data warehouse. This data warehouse must centrailize all data from my company, so the system must periodically query my databases and load the data to the warehouse."*

> *"I want to create a streaming platform, on this platform we have two types of users, the creators, responsible for create the livestreams and the watchers, the users that consume the lives. It must have fast and stable content delivery."*

However, for the sake of simplicity, the last two use cases will only be discussed in Section 4.7, the other sections concern the first requirement. The full output to all experiments can be found on this thesis repository (ROSA, 2025).

4.1 Requirements elicitation

The adapted requirements elicitation process enhanced the refinement phase to generate user-centered requirements while avoiding unnecessary technical details. The final list of requirements, generated after interactions with the user, can be found in the appendix F.

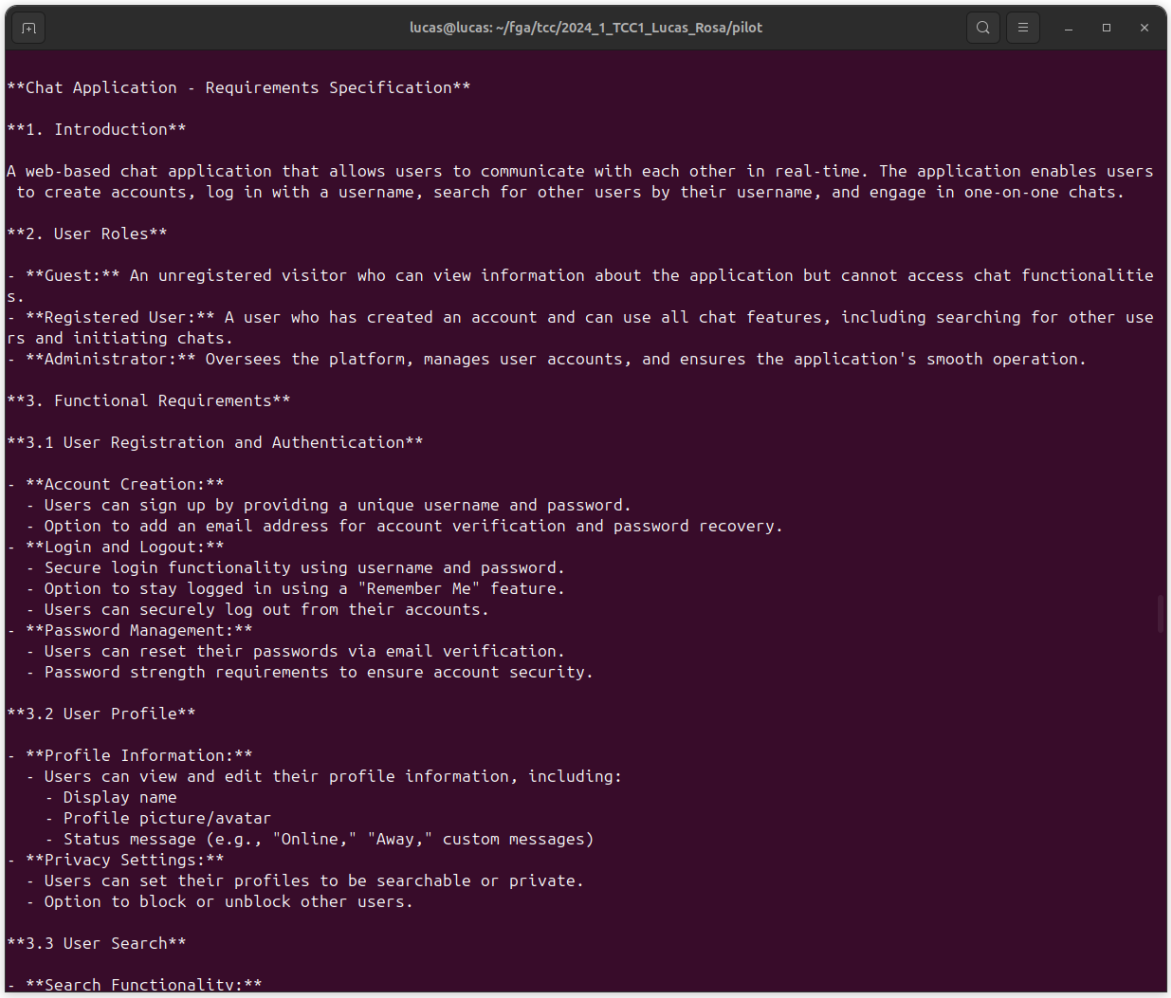


Figure 9 – Requirements elicitation result

4.2 Architecture generation

The generated requirements list was then used to produce an AWS-based architecture for the project, employing AWS CDK with Python. The architectural design aimed to utilize AWS services effectively, resulting in a modular and cloud-native solution. The full architecture can also be found on appendix G.

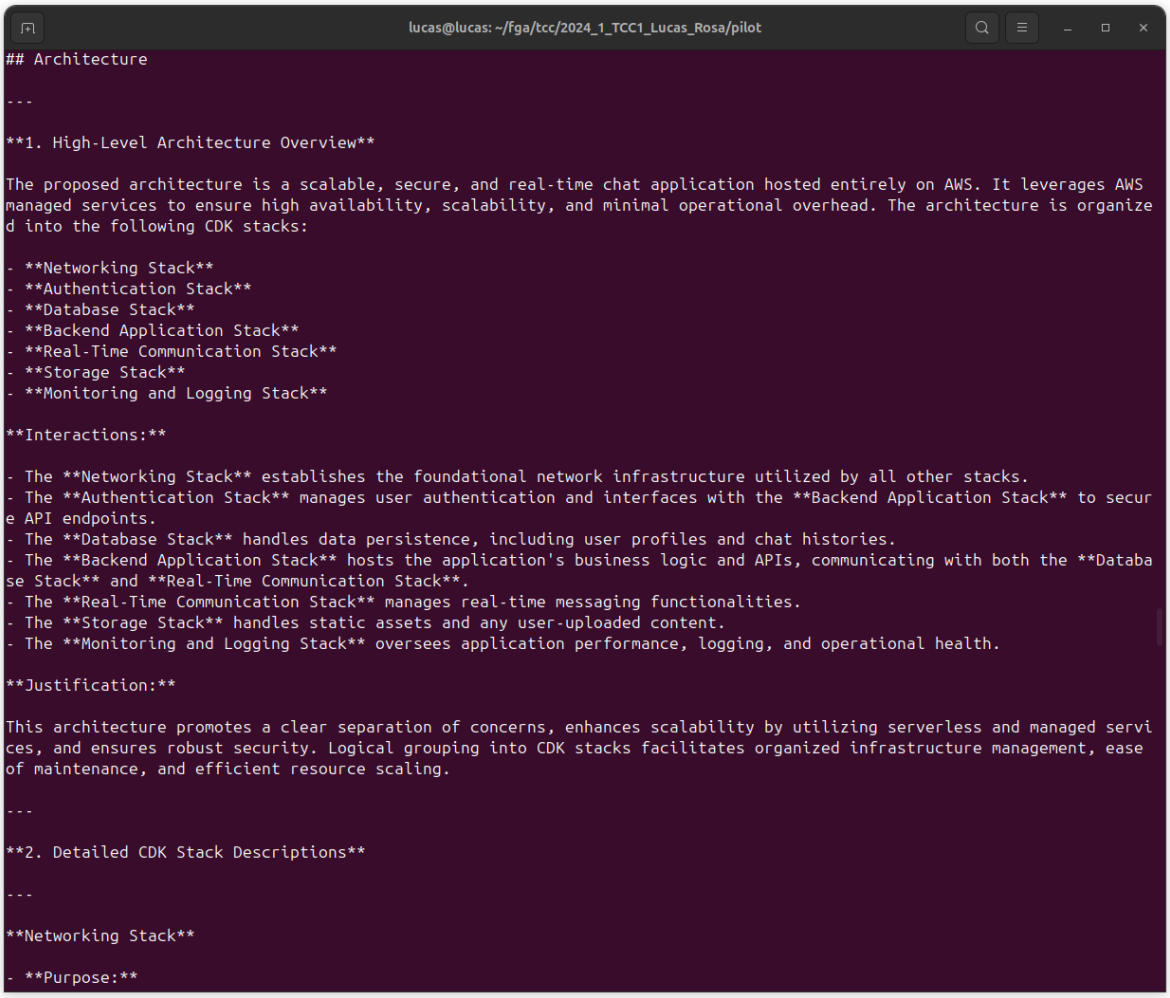


Figure 10 – Architecture result

4.3 End-to-end use cases generation

Using the application specifications and the overall architecture the system is now able to generate the end-to-end use cases description. The complete output can be found on appendix H

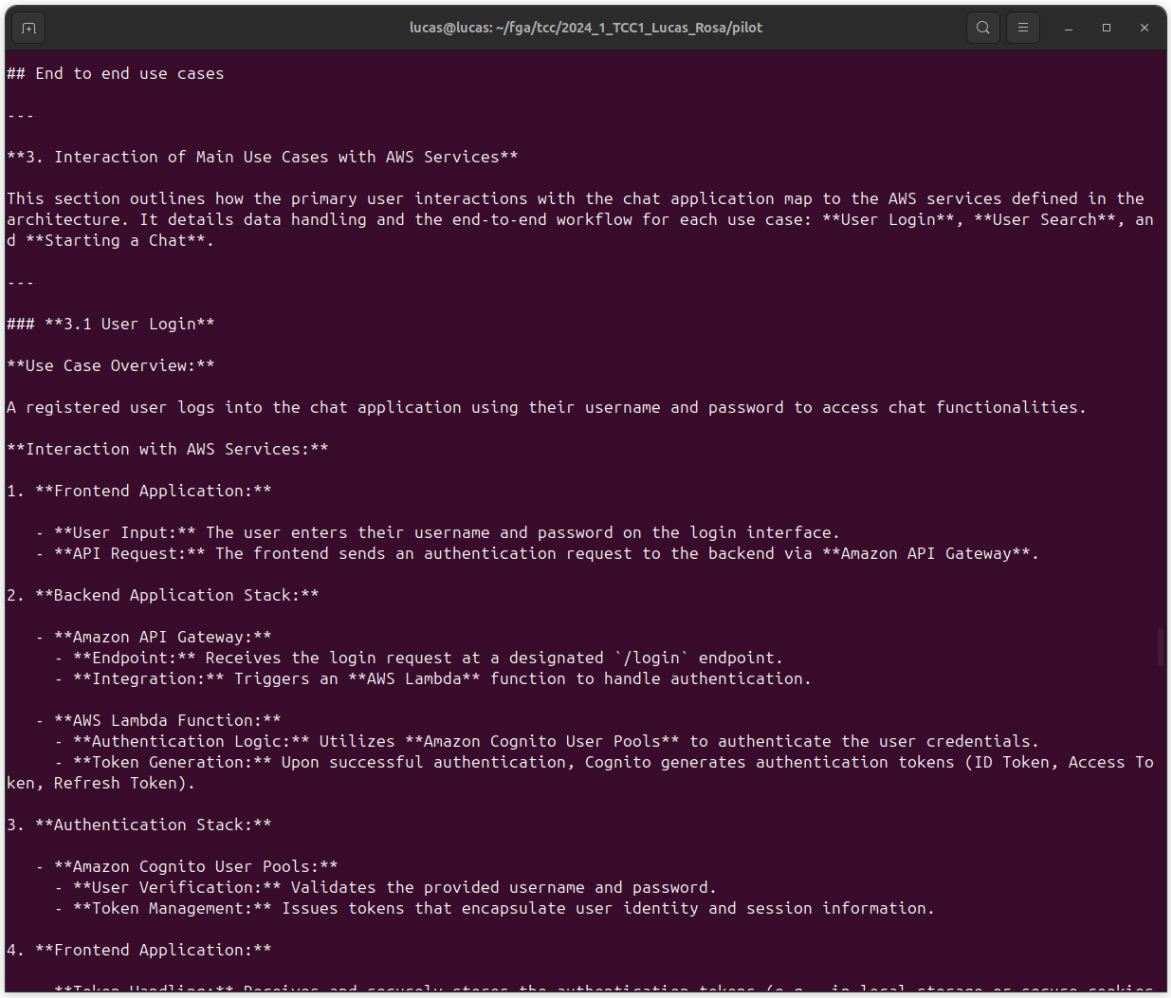


Figure 11 – End-to-end use cases result

4.4 End-to-end use cases diagrams

The next step is to generate the use case diagrams. Only the rendered images will be displayed on this section, but the mermaid source code can be found on appendix I.

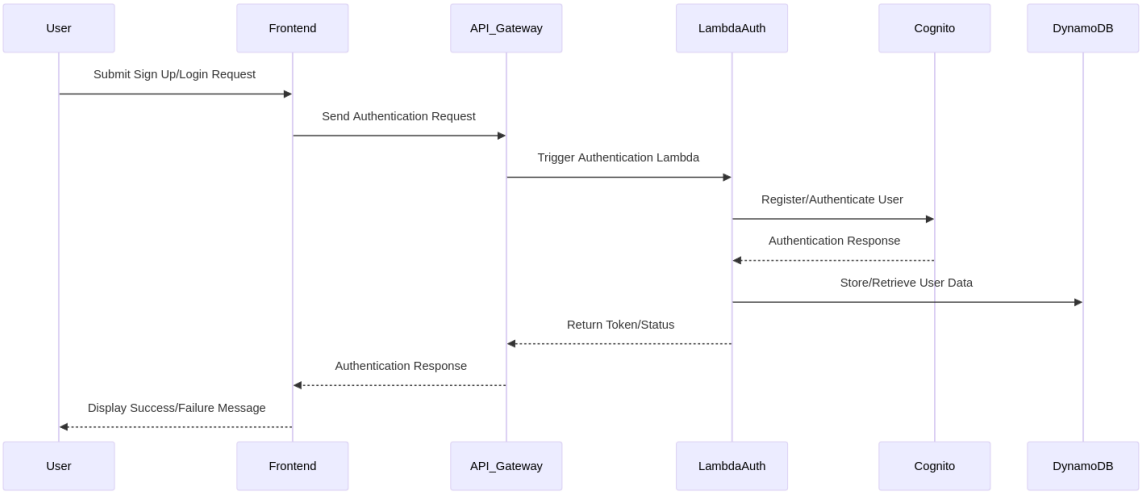


Figure 12 – User registration use case

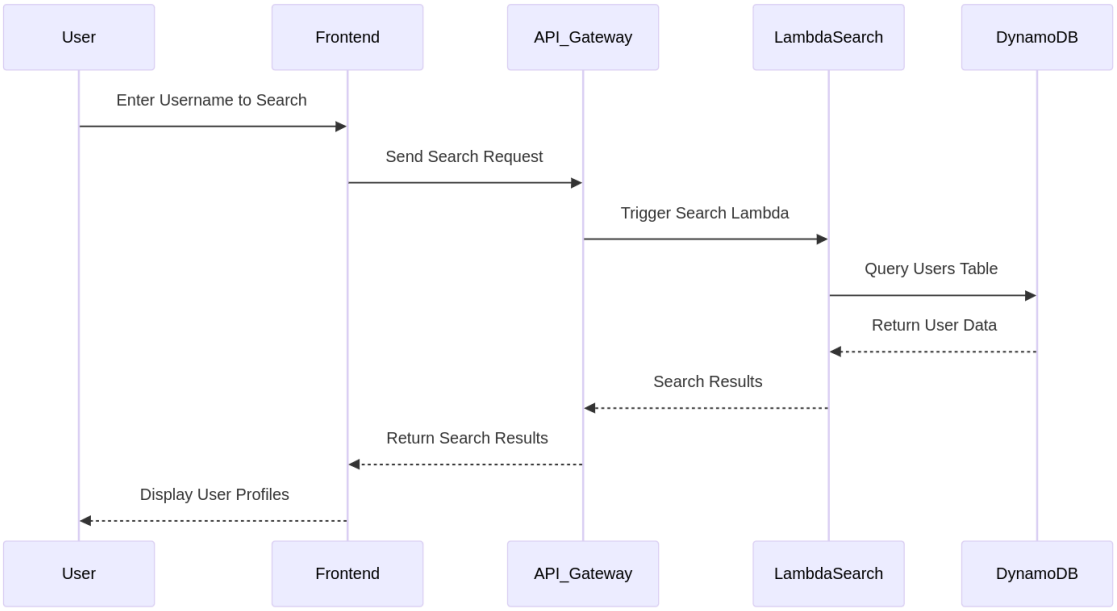


Figure 13 – Search user use case

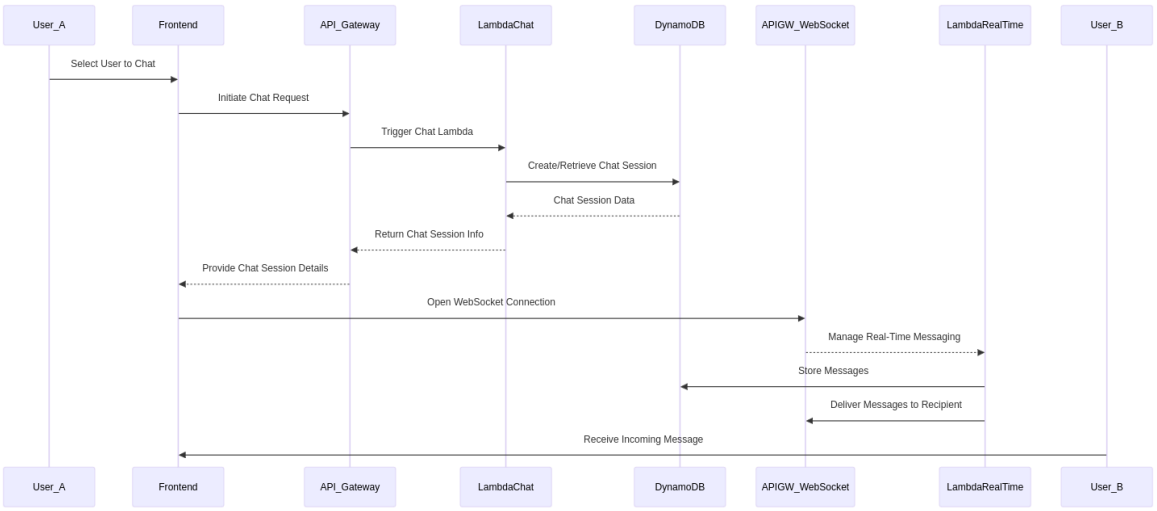


Figure 14 – Start chat use case

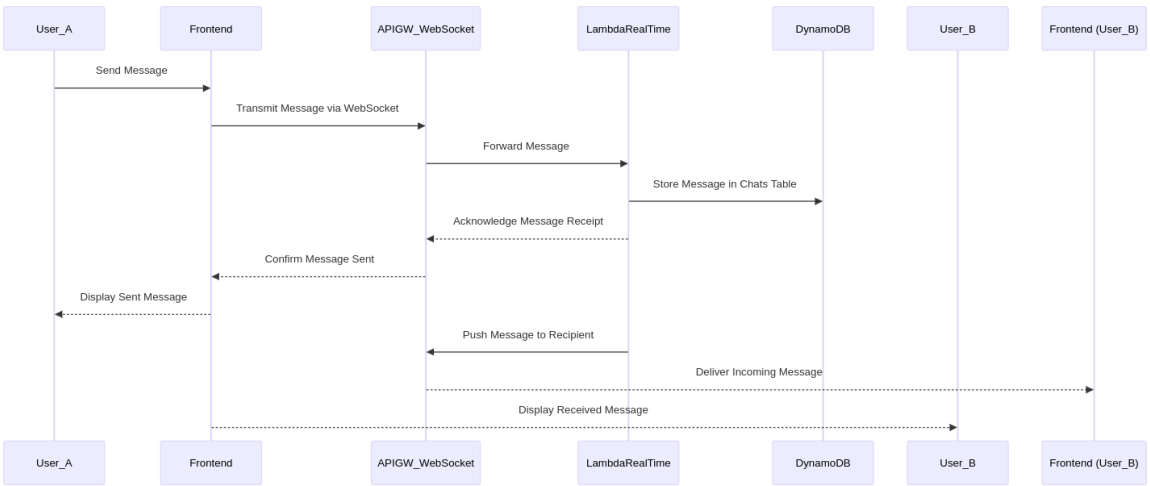


Figure 15 – Send and receive message use case

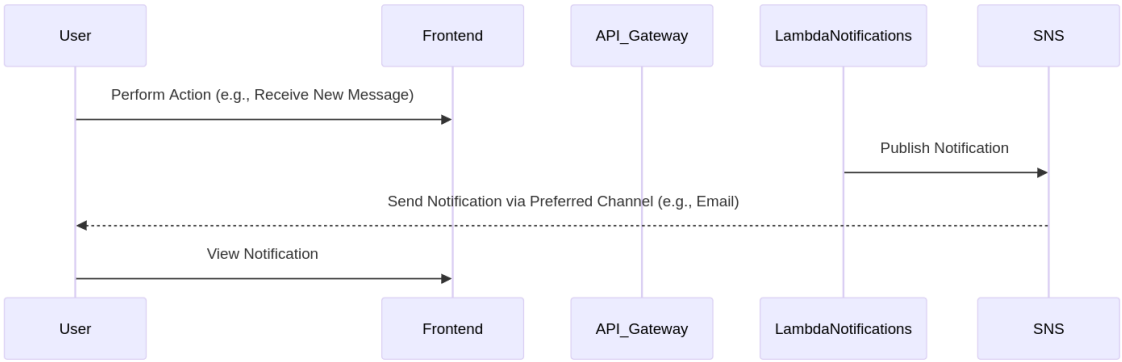


Figure 16 – Receive notification use case

4.5 Architecture diagram

The architecture was visualized using a Mermaid diagram, which illustrates the components, their interactions, and overall system flow. The generated diagram provides a clear representation of how the different AWS resources work together to meet the project's needs. Mermaid source code on appendix J

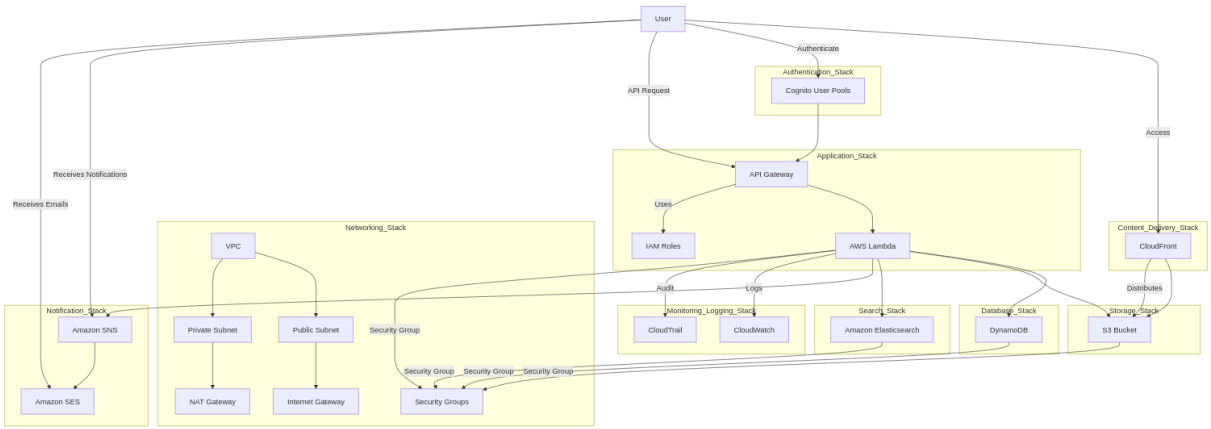


Figure 17 – Overall architecture diagram

4.6 Development plan generation

The last system step is to generate the development plain based on all data from previous steps. Appendix K for complete output.

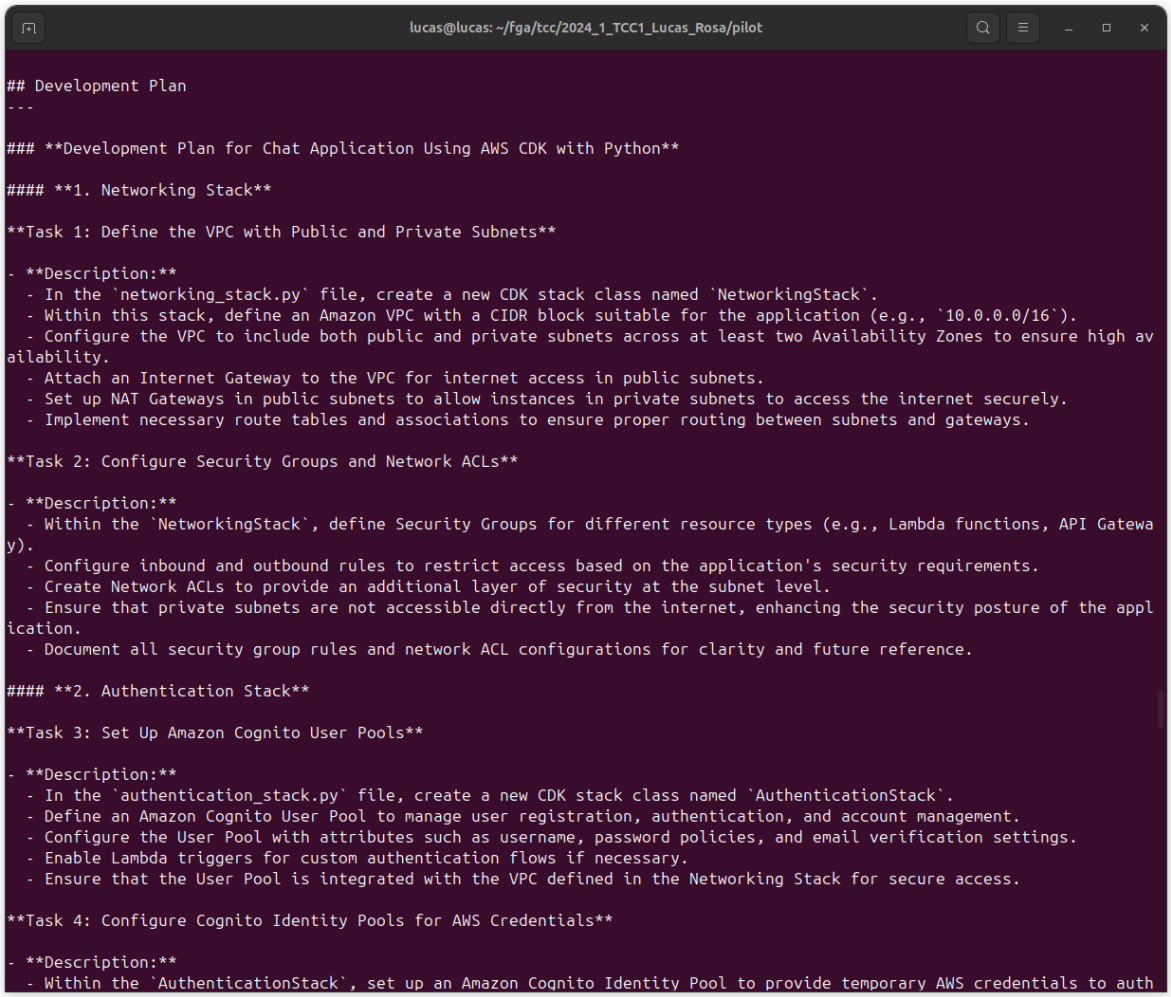


Figure 18 – Development plan result

4.7 Evaluate generated solution

To evaluate the quality of the generated solutions, we compared the overall architectures against the criteria defined in Section 3.2.7.3. Note that the outputs for the Live Streaming Platform and Data Warehouse projects have been omitted from this document; their files are available in the thesis repository (ROSA, 2025).

For this analysis, only the stacks containing resources that directly impact the project domain were evaluated. Common stacks, such as those handling networking, monitoring, or authentication, were excluded from this assessment.

4.7.1 Live chat application

4.7.1.1 Generated architecture

```
**Database Stack**

- **Purpose:**
    - Manages data persistence for user profiles and chat histories.

- **AWS Resources:**
    - **Amazon DynamoDB:** A fully managed NoSQL database for storing
    user information and chat messages.
    - **DynamoDB Tables:**
        - **Users Table:** Stores user profiles and related data.
        - **Chats Table:** Stores chat sessions and messages.
    - **DynamoDB Streams (optional):** Enables real-time data processing
    or triggering events based on data changes.

- **Interactions:**
    - The **Backend Application Stack** interacts with DynamoDB to
    perform CRUD operations on user data and chat messages.

- **Rationale:**
    - DynamoDB provides low-latency, high-throughput data storage, making
    it ideal for real-time applications like chat apps. Its scalability
    ensures that the database can handle varying loads efficiently.

**Backend Application Stack**

- **Purpose:**
    - Hosts the application's backend APIs and business logic.

- **AWS Resources:**
    - **AWS Lambda Functions:** Implements serverless functions to handle
    API requests.
    - **Amazon API Gateway:** Exposes RESTful API endpoints for frontend
    communication.
    - **AWS AppSync (optional):** Manages GraphQL APIs if required.
    - **IAM Roles and Policies:** Defines permissions for accessing other
    AWS services securely.
```

```
- **Interactions:**
    - Communicates with the **Database Stack** to access and manipulate
    user and chat data.
    - Integrates with the **Authentication Stack** to verify user
    identities and secure endpoints.

- **Rationale:**
    - Serverless architecture using Lambda and API Gateway offers
    scalability, cost-efficiency, and reduced operational management. It
    allows the application to handle varying traffic loads seamlessly.

**Real-Time Communication Stack**

- **Purpose:**
    - Manages real-time messaging capabilities for the chat application.

- **AWS Resources:**
    - **Amazon API Gateway WebSocket APIs:** Facilitates real-time,
    two-way communication between clients and the backend.
    - **AWS Lambda Functions:** Handles WebSocket connection events and
    message routing.
    - **Amazon DynamoDB or Amazon ElastiCache (Redis):** Manages session
    state and message delivery if required.

- **Interactions:**
    - Integrates with the **Backend Application Stack** to process and
    route messages in real time.

- **Rationale:**
    - WebSocket APIs in API Gateway provide a scalable solution for
    real-time communication without the need to manage persistent server
    connections manually.
```

4.7.1.2 Comparison

The generated solution correctly incorporated the expected AWS resources. WebSocket APIs handled real-time communication, AWS Lambda supported the backend APIs, and DynamoDB stored user and session data as intended.

4.7.2 Data warehouse

4.7.2.1 Generated architecture

```
**Data Storage Stack**

- **Purpose:**
    - Provides the centralized data warehouse for storing aggregated
    company data.

- **AWS Resources:**
    - **Amazon Redshift Cluster:** Serves as the primary data warehouse
    solution.
    - **Amazon S3 Buckets:** Used for data staging and archival.
    - **Amazon Redshift Spectrum:** Enables querying data directly from
    S3.
    - **AWS Glue Data Catalog:** Maintains metadata for data stored in
    Redshift and S3.

- **Interactions:**
    - Receives transformed data from the Data Integration Stack.
    - Interfaces with reporting and analysis tools for data access.

- **Rationale:**
    - Amazon Redshift offers high-performance data warehousing
    capabilities, scalable storage, and seamless integration with AWS
    data services, meeting the need for a robust and centralized data
    repository.

**Data Integration Stack**

- **Purpose:**
    - Manages the ETL processes required to extract data from source
    databases, transform it, and load it into the data warehouse.

- **AWS Resources:**
    - **AWS Glue ETL Jobs:** Automates data extraction, transformation,
    and loading.
    - **AWS Lambda Functions:** Orchestrates and schedules ETL workflows.
    - **AWS Step Functions:** Coordinates complex ETL pipelines.
```

```
    - **AWS Glue Crawlers:** Discovers and catalogs data schemas from
    source databases.
    - **Amazon RDS Data Sources:** Connectors for various source
    databases.

- **Interactions:**
    - Connects to source databases to extract data periodically.
    - Loads processed data into the Amazon Redshift cluster within the
    Data Storage Stack.
    - Utilizes the Networking Stack's VPC for secure data transfer.

- **Rationale:**
    - AWS Glue provides a scalable and managed ETL service, reducing the
    need for custom data pipeline development. Integration with Lambda
    and Step Functions enables flexible and automated ETL workflows,
    aligning with the requirement for periodic data loading.

**Security Stack**

- **Purpose:**
    - Ensures the security and compliance of the data warehouse
    environment.

- **AWS Resources:**
    - **AWS Identity and Access Management (IAM):** Manages user
    permissions and roles.
    - **AWS Key Management Service (KMS):** Handles encryption keys for
    data encryption.
    - **AWS Secrets Manager:** Stores and manages database credentials
    securely.
    - **AWS Shield and AWS WAF:** Protects against DDoS attacks and web
    exploits.
    - **Amazon Macie:** Discovers, classifies, and protects sensitive
    data.
    - **AWS Config:** Monitors and records AWS resource configurations
    for compliance.

- **Interactions:**
    - Applied across all stacks to control access and secure data flows.
```

```
    - Integrates with the Data Integration and Data Storage stacks to
    enforce encryption and access policies.

- **Rationale:**
    - Leveraging AWS security services provides comprehensive protection,
    ensuring data confidentiality, integrity, and compliance with
    regulatory standards. Centralizing security in a dedicated stack
    enforces consistent security practices across the architecture.
```

4.7.2.2 Comparison

The generated architecture successfully utilized Amazon S3 for the data lake, Amazon Redshift for the data warehouse, and AWS Glue for ETL processes. However, it did not address some key components such as real-time data streaming with Kinesis, ad-hoc querying with Athena, or data visualization with QuickSight.

4.7.3 Live streaming platform

4.7.3.1 Generated architecture

```
**Streaming Stack**

- **Purpose:**

  Handles live stream ingestion, processing, and delivery to ensure
  low-latency and high-quality streaming experiences.

- **AWS Resources:**

  - **Amazon Interactive Video Service (IVS):** Manages live stream
  ingestion and real-time streaming.
  - **Amazon CloudFront:** Distributes live and recorded streams globally
  with low latency.
  - **AWS Elemental MediaLive (optional):** For advanced live video
  processing needs.
  - **Amazon Route 53:** Manages DNS routing for streaming endpoints.

- **Interactions:**

  - Receives live streams from **Creators** via the **Frontend Hosting
  Stack**.
```

```
  - Distributes streams to **Watchers** through **CloudFront**.
  - Interfaces with the **Storage and Content Delivery Stack** for
  archived content.

- **Rationale:**

  Leveraging Amazon IVS ensures a managed, scalable, and reliable
  streaming solution tailored for interactive live video, minimizing
  latency and enhancing viewer experience.

**Storage and Content Delivery Stack**

- **Purpose:**

  Manages media storage, archival of recorded streams, and efficient
  content distribution to users.

- **AWS Resources:**

  - **Amazon S3:** Stores recorded livestreams, user-generated content,
  and static assets.
  - **Amazon CloudFront:** Acts as the CDN for delivering stored content
  with low latency.
  - **AWS Elemental MediaConvert (optional):** Transcodes recorded
  streams into various formats for compatibility.
  - **AWS Transfer for SFTP (optional):** Enables secure file transfers
  if needed.

- **Interactions:**

  - Receives recorded streams from the **Streaming Stack** for archival.
  - Serves stored content to the **Frontend Hosting Stack** and directly
  to **Watchers**.
  - Integrates with the **Application Backend Stack** for managing media
  metadata.

- **Rationale:**
```

```
Amazon S3 combined with CloudFront offers a robust and scalable
solution for storing and delivering media content, ensuring high
availability and performance.
```

4.7.3.2 Comparison

The generated architecture for the live streaming platform effectively addressed all major design requirements. It leverages Amazon IVS for scalable and reliable streaming, incorporates MediaLive for video processing, and uses CloudFront to ensure efficient content delivery.

4.8 Summary of results

The methodology applied successfully transformed user requirements into comprehensive, cloud-based solutions using LLM agents and AWS CDK. The key outcomes are as follows:

- A refined requirements elicitation process that produced clear, user-focused requirements.
- Modular AWS architectures that leverage cloud-native services for scalability, reliability, and cost efficiency.
- Detailed architectural diagrams, generated using Mermaid, that enhance clarity and facilitate stakeholder communication.
- Successful implementation of expected core components in the Live Chat Application, while the Data Warehouse and Live Streaming Platform architectures demonstrated strengths in resource allocation and workflow design, despite some omissions (e.g., Kinesis for streaming, Athena for querying, and QuickSight for visualization in the Data Warehouse).

Overall, the evaluation confirms that the agent-generated solutions are closely aligned with AWS reference architectures and best practices, indicating a strong performance in transforming user requirements into functional cloud architectures.

Bibliography

Amazon. *Tutorial: Create a WebSocket chat app with a WebSocket API, Lambda and DynamoDB - Amazon API Gateway*. Disponível em: <https://docs.aws.amazon.com/apigateway/latest/developerguide/websocket-api-chat-app.html>. Citado na página 45.

Amazon. *Video streaming architectures*. Disponível em: <https://aws.amazon.com/developer/application-security-performance/articles/video-streaming-architectures/>. Citado 2 vezes nas páginas 46 and 47.

Amazon. *Integrate Industrial Data with Enterprise Systems in AWS Cloud*. 2024. Disponível em: <https://d1.awsstatic.com/architecture-diagrams/ArchitectureDiagrams/highbyte-intelligence-hub-industrial-dataops-on-aws-ra.pdf?did=wp_card&trk=wp_card>. Citado na página 46.

BENDER, E. M. et al. On the dangers of stochastic parrots: Can language models be too big? . In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. New York, NY, USA: Association for Computing Machinery, 2021. (FAccT '21), p. 610–623. ISBN 9781450383097. Disponível em: <https://doi.org/10.1145/3442188.3445922>. Citado na página 25.

BROWN, T. B. et al. *Language Models are Few-Shot Learners*. 2020. Disponível em: <https://arxiv.org/abs/2005.14165>. Citado na página 25.

DEVLIN, J. et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. 2019. Disponível em: <https://arxiv.org/abs/1810.04805>. Citado na página 25.

GAO, T.; FISCH, A.; CHEN, D. *Making Pre-trained Language Models Better Few-shot Learners*. 2020. Disponível em: <https://arxiv.org/abs/2012.15723v2>. Citado 2 vezes nas páginas 26 and 39.

JINJA2: A very fast and expressive template engine. Citado na página 48.

LI, X. L.; LIANG, P. *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. Disponível em: <https://arxiv.org/abs/2101.00190v1>. Citado na página 26.

LIU, P. et al. *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing*. Disponível em: <https://arxiv.org/abs/2107.13586v1>. Citado na página 26.

OPENAI. *OpenAI API*. 2021. Disponível em: <https://beta.openai.com/docs/>. Citado na página 27.

PYTHAGORA-IO. *GPT Pilot*. 2024. Accessed: 2024-06-30. Disponível em: <https://github.com/Pythagora-io/gpt-pilot>. Citado 2 vezes nas páginas 26 and 27.

ROSA, L. *2024_1_TCC1_Lucas_Rosa/executions_output at main · medeiroslucas/2024_1_TCC1_Lucas_Rosa*. 2025. Disponível em: <https://github.com/medeiroslucas/2024_1_TCC1_Lucas_Rosa/tree/main/executions_output>. Citado 2 vezes nas páginas 49 and 56.

Synergy Research Group. *The Decade's Megatrends in Numbers – Part 1: Cloud Goes from 0 to 100 in Ten Years while Enterprise Data Center Spending Stagnates | Synergy Research Group*. 2020. Disponível em: <https://www.srgresearch.com/articles/the-decades-megatrends-in-numbers-part-1>. Citado na página 21.

WHAT is AWS? - Cloud Computing with AWS - Amazon Web Services. 2024. Disponível em: <https://aws.amazon.com/what-is-aws/>. Citado na página 27.

XU, A. *EP74: The Evolution of AWS Services*. 2023. Disponível em: <https://blog.bytebytego.com/p/ep74-the-evolution-of-aws-services>. Citado na página 22.

Appendix

APPENDIX A – GPTPilot requirements elicitation prompt

```
Your task is to talk to a new client and develop a detailed specification
for a new application the client wants to build. This specification will
serve as an input to an AI software developer and thus must be very
detailed, contain all the project functionality and precisely define
behaviour, 3rd-party integrations (if any), etc.

The AI developer prefers working on web apps using
Node/Express/MongoDB/Mongoose/EJS stack, and use vanilla JS with
Bootstrap on the frontend, unless the client has different requirements.
Try to avoid the use of Docker, Kubernetes, microservices and single-page
app frameworks like React, Next.js, Angular, Vue or Svelte unless the
brief explicitly requires it.

In your work, follow these important rules:
* In your communication with the client, be straightforward, concise, and
focused on the task.
* Ask questions ONE BY ONE. This is veryy important, as the client is
easily confused. If you were to ask multiple questions the user would
probably miss some questions, so remember to always ask the questions one
by one
* Ask specific questions, taking into account what you already know about
the project. For example, don't ask "what features do you need?" or
"describe your idea"; instead ask "what is the most important feature?"
* Pay special attention to any documentation or information that the
project might require (such as accessing a custom API, etc). Be sure to
ask the user to provide information and examples that the developers will
need to build the proof-of-concept. You will need to output all of this
in the final specification.
* This is a a prototype project, it is important to have small and
well-defined scope. If the scope seems to grow too large (beyond a week
or two of work for one developer), ask the user if they can simplify the
project.
```

```
* Do not address non-functional requirements (performance, deployment,
security, budget, timelines, etc...). We are only concerned with
functional and technical specification here.
* Do not address deployment or hosting, including DevOps tasks to set up
a CI/CD pipeline
* Don't address or invision any future development (post
proof-of-concept), the scope of your task is to only spec the
PoC/prototype.
* If the user provided specific information on how to access 3rd party
API or how exactly to implement something, you MUST include that in the
specification. Remember, the AI developer will only have access to the
specification you write.

Ensure that you have all the information about:
* overall description and goals for the app
* all the features of the application
* functional specification
    * how the user will use the app
    * enumerate all the parts of the application (eg. pages of the
    application, background processing if any, etc); for each part,
    explain *in detail* how it should work from the perspective of the
    user
    * identify any constraints, business rules, user flows or other
    important info that affect how the application works or how it is
    used
* technical specification
    * what kind of an application this is and what platform/technologies
    will be used
    * the architecture of the application (what happens on backend,
    frontend, mobile, background tasks, integration with 3rd party
    services, etc)
    * detailed description of each component of the application
    architecture
* integration specification
    * any 3rd party apps, services, APIs that will be used (eg. for auth,
    payments, etc..)
    * if a custom API is used, precise definitions, with examples, how to
    use the custom API or do the custom integration
```

```
If you identify any missing information or need clarification on any
vague or ambiguous parts of the brief, ask the client about it.

Important note: don't ask trivial questions for obvious or unimportant
parts of the app, for example:
* Bad questions example 1:
  * Client brief: I want to build a hello world web app
  * Bad questions:
    * What title do you want for the web page that displays "Hello
    World"?
    * What color and font size would you like for the "Hello World" text
    to be displayed in?
    * Should the "Hello World" message be static text served directly
    from the server, or would you like it implemented via JavaScript on
    the client side?
  * Explanation: There's no need to micromanage the developer(s) and
  designer(s), the client would've specified these details if they were
  important.

If you ask such trivial questions, the client will think you're stupid
and will leave. DOn'T DO THAT

Think carefully about what a developer must know to be able to build the
app. The specification must address all of this information, otherwise
the AI software developer will not be able to build the app.

When you gather all the information from the client, output the complete
specification. Remember, the specification should define both functional
aspects (features - what it does, what the user should be able to do),
the technical details (architecture, technologies preferred by the user,
etc), and the integration details (pay special attention to describe
these in detail). Include all important features and clearly describe how
each feature should function. IMPORTANT: Do not add any preamble (eg.
"Here's the specification....") or conclusion/commentary (eg. "Let me
know if you have further questions")!

Here's an EXAMPLE initial prompt:
---start-of-example-output---
```

```
Online forum similar to Hacker News (news.ycombinator.com), with a simple
and clean interface, where people can post links or text posts, and other
people can upvote, downvote and comment on. Reading is open to anonymous
users, but users must register to post, upvote, downvote or comment. Use
simple username+password authentication. The forum should be implemented
in Node.js with Express framework, using MongoDB and Mongoose ORM.

The UI should use EJS view engine, Bootstrap for styling and plain
vanilla JavaScript. Design should be simple and look like Hacker News,
with a top bar for navigation, using a blue color scheme instead of the
orange color in HN. The footer in each page should just be "Built using
GPT Pilot".

Each story has a title (one-line text), a link (optional, URL to an
external article being shared on AI News), and text (text to show in the
post). Link and text are mutually exclusive - if the submitter tries to
use both, show them an error.

Use the following algorithm to rank top stories, and comments within a
story: "score = upvotes - downvotes + comments - sqrt(age)" , where
"upvotes" and "downvotes" are the number of upvotes and downvotes the
story or comment has, "comments" is the number of comments for a story
(total), or the number of sub-comments (for a comment), and "age" is how
old is the story, in minutes, and "sqrt" is the square root function.

Implement the following pages:

* / - shows the top 20 posted stories, ranked using the scoring
algorithm, with a "More" link that shows the next 20 (pagination using
"p" query parameter), and so on
* /newest - shows the latest 20 posted stories, ranked chronologically
(newest first), with a "More" link that shows the next 20 (pagination
using "p" query parameter), and so on
* /submit - shows a form to submit a new story, upon submitting the user
should get redirected to /newest
* /login - shows a login form (username, password, "login" button, and a
link to register page for new users)
* /register - shows a register form (username, password, "register"
button, and a link to login page for existing users)
```

* /item - shows the story (use "id" query parameter to pass the story ID to this route)
* /comment - shows the form to send a comment (just a textarea and "submit" button) - upon commenting, the person should get redirected to the story they commented on

The / and /newest pages should show the story title (link to the external article if "link" is set, otherwise link to the story item /item page), number of points (points = upvotes - downvotes), poster username (no link), how old is the story ("x minutes ago", "y hours ago" or "z days ago"), and "xyz comments" (link to /item page of the story). This is basically the same how HN shows it.

The /item page should also follow the layout for HN in how it shows the story, and the comments tree. Instead of the embedded "reply" form, the story should just have a "comment" button that goes to the /comment page, similar to the "reply" link underneath each comment. Both should link to the /comment page.
---end-of-example-output---

Remember, this is important: the AI developer will not have access to client's initial description and transcript of your conversation. The developer will only see the specification you output on the end. It is very important that the spec captures *all* the details of the project in as much detail and precision as possible.

Note: after the client reads the specification you create, the client might have additional comments or suggestions. In this case, continue the discussion with the user until you get all the new information and output the newly updated spec again.

APPENDIX B – Custom requirements elicitation prompt

```
**Your Task:**

Engage with a new client to develop a comprehensive and detailed
requirements specification for their new web application.
This specification will serve as the sole input for the development team,
so it must accurately capture all the functional aspects and user needs
without addressing technical implementation details.

**Guidelines for Interaction:**

- **Communication Style:**
  - Be straightforward, concise, and focused on understanding the
  client's requirements.
  - Maintain a professional and empathetic tone to encourage open
  communication.

- **Questioning:**
  - You may ask up to **7 follow-up questions**; prioritize those that
  gather essential information.
  - Ask questions **one at a time** to avoid overwhelming the client.
  - Focus on **specific, impactful questions** that clarify the client's
  needs.
  - **Avoid trivial questions** about minor details unless they
  significantly impact functionality.

- **Scope Management:**
  - Ensure the project's scope is manageable (approximately one to two
  weeks of development work).
  - If the scope becomes too large, assist the client in simplifying the
  requirements.

**Information to Gather:**
```

```
- **Overall Description and Goals:**
  - Understand the main objectives and desired outcomes of the
  application.

- **Features and Functionality:**
  - Identify all required features and how they benefit the user.
  - Gather details on user interactions, workflows, and any business
  rules.
  - Clarify any constraints or specific requirements that affect how the
  application works.

- **User Experience:**
  - Understand how the client envisions the user interface and
  interactions.
  - Inquire about any specific user roles and permissions.

- **Content and Data:**
  - Determine what content or data will be displayed or processed.
  - Identify any integrations with external data sources (without delving
  into technical integration details).

**Final Requirements Specification:**

- **Comprehensive Detail:**
  - Provide a complete requirements specification that includes all
  gathered information.
  - Use clear headings, subheadings, and bullet points for readability.

- **Functional Requirements:**
  - Describe what the application does and what users can achieve.
  - Detail each feature's functionality from the user's perspective.
  - Include user stories or use cases to illustrate how users will
  interact with the application.

- **No Technical Details:**
  - Focus solely on requirements without specifying technical solutions
  or technologies.

- **No Preamble or Conclusion:**
```

```
  - Present the requirements directly without additional commentary or
  salutations.

**Additional Notes:**

- **Iterative Updates:**
  - If the client provides new information or changes, update the entire
  requirements specification accordingly.
  - Ensure the updated specification is complete and reflects all
  modifications.

- **Assumptions:**
  - If you make any assumptions, clearly state them in the requirements.

---

**Example of Transforming a User Description into Requirements:**

---

**Client's Description:**

"I want to build a web application for booking fitness classes at local
gyms. Users should be able to search for classes by type (like yoga,
pilates, or spinning), schedule, and location. They should be able to
create an account, book a class, and receive a confirmation email. Gyms
should be able to list their classes, manage bookings, and see who has
signed up."

---

**Transformed Requirements Specification:**

---

**Fitness Class Booking Web Application - Requirements Specification**

**1. Introduction**
```

```
A web-based platform that connects users with fitness classes offered by
local gyms. The application allows users to search and book classes,
while enabling gyms to manage class listings and bookings.

**2. User Roles**

- **Visitor:** An unregistered user who can browse classes.
- **User:** A registered individual who can book classes.
- **Gym Owner/Admin:** A representative of a gym who can manage class
listings and view bookings.
- **Administrator:** Platform administrator overseeing operations and
user management.

**3. Functional Requirements**

**3.1 User Registration and Authentication**

- **Account Creation for Users:**
  - Users can sign up using an email address and password.
  - Option to register via social media accounts (e.g., Facebook, Google)
  for quicker sign-up.
- **Account Creation for Gyms:**
  - Gyms can register by providing gym details and contact information.
  - Gym accounts require admin approval before they can list classes.
- **Login and Logout:**
  - Secure login and logout functionality for all registered users.
- **Profile Management:**
  - Users can update personal information and view booking history.
  - Gyms can update gym information and manage class offerings.

**3.2 Class Listings (Gym Owner/Admin)**

- **Create and Manage Classes:**
  - Gyms can add new classes with details:
    - Class name
    - Description
    - Class type/category (e.g., Yoga, Pilates)
    - Instructor name
    - Location/address
```

```
    - Schedule (date and time)
    - Duration
    - Capacity (maximum number of participants)
    - Price
- **Edit and Cancel Classes:**
  - Gyms can modify class details or cancel classes.
  - Users who have booked a canceled class receive notifications and
  refunds if applicable.
- **View Bookings:**
  - Gyms can see a list of users who have booked each class.

**3.3 Class Search and Browsing (User and Visitor)**

- **Search Functionality:**
  - Users can search for classes using keywords.
- **Filter Options:**
  - Filter classes by:
    - Class type/category
    - Date and time
    - Location (city, zip code)
    - Price range
    - Gym name
- **Class Listings:**
  - Display key class information in search results:
    - Class name
    - Gym name
    - Class type
    - Date and time
    - Price
- **Class Details Page:**
  - Provides comprehensive details about the class.
  - Includes a "Book Now" button (for registered users).

**3.4 Booking Classes (User)**

- **Booking Process:**
  - Users can book a class from the class details page.
  - Prompt to log in or register if not already authenticated.
- **Payment Processing:**
```

```
  - Secure payment gateway to handle class fees.
  - Accept major credit cards and PayPal.
- **Booking Confirmation:**
  - Users receive an on-screen confirmation and a confirmation email with
  booking details.
- **Booking Management:**
  - Users can view upcoming bookings in their profile.
  - Option to cancel a booking within a specified time frame (e.g., 24
  hours before class).

**3.5 Notifications**

- **Email Notifications:**
  - Sent to users for booking confirmations, cancellations, and reminders
  24 hours before the class.
  - Sent to gyms when a new booking is made or a booking is canceled.
- **On-site Notifications:**
  - Dashboard alerts for gyms about upcoming classes and bookings.
  - Notifications for users about class changes or important updates.

**3.6 Reviews and Ratings**

- **Leave Reviews:**
  - After attending a class, users can rate the class and leave a review.
- **Display Ratings:**
  - Class listings show average ratings and excerpts from recent reviews.
- **Review Moderation:**
  - Gyms can respond to reviews.
  - Administrators can remove inappropriate content.

**3.7 Administrative Functions**

- **User Management:**
  - Administrators can manage all user accounts (users and gyms).
  - Ability to suspend or deactivate accounts violating terms of service.
- **Content Management:**
  - Oversee class listings and remove inappropriate content.
- **Reporting and Analytics:**
```

```
  - Access to platform usage statistics, such as total bookings, active
  users, popular classes, etc.

**3.8 Security and Data Protection**

- **User Data Privacy:**
  - Compliance with relevant data protection regulations (e.g., GDPR).
  - Secure storage of personal and payment information.
- **Payment Security:**
  - Use of SSL encryption for transactions.
  - Compliance with PCI DSS standards.

**4. User Interface Requirements**

- **Responsive Design:**
  - The web application should be accessible and optimized for desktop,
  tablet, and mobile browsers.
- **User-Friendly Navigation:**
  - Intuitive menus and navigation bars for easy access to features.
- **Search and Filter Accessibility:**
  - Prominently placed search bar and filter options on the classes page.
- **Visual Design:**
  - Clean and modern aesthetic with high-quality images for class
  listings.
- **Accessibility:**
  - Features to assist users with disabilities (e.g., alt text for
  images, keyboard navigation).

**5. Non-Functional Requirements**

*(Note: While the focus is on requirements, this section avoids technical
details and remains high-level.)*

- **Performance:**
  - Fast loading times for all pages to enhance user experience.
- **Scalability:**
  - Ability to handle an increasing number of users and classes without
  performance degradation.
- **Reliability:**
```

```
  - High availability with minimal downtime.

**6. Assumptions**

- All users have internet access and a modern web browser.
- Gyms are responsible for the accuracy of their class listings and
handling on-site class logistics.
- Payment processing fees are managed within the platform's business
model (details to be defined).

**7. Constraints**

- Users must agree to terms and conditions during account creation.
- Cancellations and refunds are subject to the gym's policies, which
should be clearly stated in class listings.

---

This requirements specification translates the client's description into
detailed, user-focused requirements for the web application. It outlines
the necessary features and functionalities without specifying technical
implementation details, ensuring that the development team understands
what needs to be built from the user's perspective.

---

Following I am sending you the user description of hes application,
generate the specifications:
```

APPENDIX C – GPTPilot architecture generation prompt

```
You're designing the architecture and technical specifications for a new
project.

If the project requirements call out for specific technology, use that.
Otherwise, if working on a web app, prefer Node.js for the backend (with
Express if a web server is needed, and MongoDB if a database is needed),
and Bootstrap for the front-end. You MUST NOT use Docker, Kubernetes,
microservices and single-page app frameworks like React, Next.js,
Angular, Vue or Svelte unless the project details explicitly require it.

Here are the details for the new project:
------------------------------
{{ project_details }}
{{ features_list }}
------------------------------

Based on these details, think step by step to design the architecture for
the project and choose technologies to use in building it.

1. First, design and describe project architecture in general terms
2. Then, list any system dependencies that should be installed on the
system prior to start of development.  For each system depedency, output
a {{ os }} command to check whether it's installed.
3. Finally, list any other 3rd party packages or libraries that will be
used (that will be installed later using packager a package manager in
the project repository/environment).
4. {% if templates %}Optionally, choose a project starter template.{%
else %}(for this project there are no available starter/boilerplate
templates, so there's no template to choose){% endif %}
```

```
{% if templates %}You have an option to use a project template that
implements standard boilerplate/scaffolding so you can start faster and
be more productive. To be considered, a template must be compatible with
the architecture and technologies you've choosen (it doesn't need to
implement everything that will be used in the project, just a useful
subset). If multiple templates can be considered, pick one that's the
best match.

If no project templates are a good match, don't pick any! It's better to
start from scratch than to use a template that is not a good fit for the
project and then spend time reworking it to fit the requirements.

Here are the available project templates:
{% for name, tpl in templates.items() %}
### {{ name }}
{{ tpl['description']}}

Contains:
{{ tpl['summary']}}
{% endfor %}{% endif %}

*IMPORTANT*: You must follow these rules while creating your project:

* You must only list *system* dependencies, ie. the ones that need to be
installed (typically as admin) to set up the programming language,
database, etc. Any packages that will need to be installed via
language/platform-specific package managers are *not* system
dependencies.
* If there are several popular options (such as Nginx or Apache for web
server), pick one that would be more suitable for the app in question.
* DO NOT include text editors, IDEs, shells, OpenSSL, CLI tools such as
git, AWS, or Stripe clients, or other utilities in your list. only direct
dependencies required to build and run the project.
* If a dependency (such as database) has a cloud alternative or can be
installed on another computer (ie. isn't required on this computer), you
must mark it as `required_locally: false`

Output only your response in JSON format like in this example, without
other commentary:
```

```json
{
    "architecture": "Detailed description of the architecture of the
    application",
    "system_dependencies": [
        {
            "name": "Node.js",
            "description": "JavaScript runtime for building apps. This is
            required to be able to run the app you're building.",
            "test": "node --version",
            "required_locally": true
        },
        {
            "name": "MongoDB",
            "description": "NoSQL database. If you don't want to install
            MongoDB locally, you can use a cloud version such as MongoDB
            Atlas.",
            "test": "mongosh --version",
            "required_locally": false
        },
        ...
    ],
    "package_dependencies": [
        {
            "name": "express",
            "description": "Express web server for Node"
        },
        ...
    ],
    "template": "name of the project template to use" // or null if you
    decide not to use a project template
}
```

APPENDIX D – Custom architecture generation prompt

```
**Your Task:**

You are designing the architecture and technical specifications for a new
project that will be entirely hosted on AWS, utilizing appropriate AWS
services. The architecture will be implemented using AWS CDK with Python.
Your goal is to create an architecture composed of CDK stacks, where each
stack represents a logical grouping of related application components
(e.g., networking, databases, authentication), rather than being based on
specific use cases.

**Project Details:**

------------------------------
**Client Brief:**

{{ project_description }}

**Project Specification:**

{{ project_specs }}
------------------------------

**Instructions:**

Based on these details, design the architecture step by step, choosing
AWS services that best meet the project requirements.

1. **High-Level Architecture Overview:**

   - **Description:**
     - Provide a general overview of the proposed architecture.
     - Explain the overall structure and how it addresses the project
     needs.
```

```
   - **CDK Stacks List:**
     - List all the CDK stacks you plan to use.
   - **Interactions:**
     - Describe how the stacks will interact with each other.
   - **Justification:**
     - Explain why this architecture is the best option for this project.

2. **Detailed CDK Stack Descriptions:**

   For each CDK stack, provide:

   - **Stack Name:**
     - A clear and descriptive name for the stack.
   - **Purpose:**
     - A brief description of the stack's role in the architecture.
   - **AWS Resources:**
     - List the AWS services and resources that will be included in this
     stack.
     - Include any relevant configurations or features.
   - **Interactions:**
     - Describe how this stack interacts with other stacks or AWS
     services.
   - **Rationale:**
     - Justify the selection of AWS services for this stack, explaining
     how they meet the project requirements.

3. **Assumptions and Considerations:**

   - **Assumptions:**
     - Note any assumptions you've made in your architectural design.
   - **Constraints:**
     - Mention any constraints that influenced your design decisions.
   - **Scalability and Security:**
     - Briefly address how the architecture accommodates scalability and
     security needs.

**Formatting Guidelines:**

- Use clear headings and subheadings to organize your output.
```

```
- Present information in bullet points or numbered lists where
appropriate.
- Be concise but provide sufficient detail for clarity.
- Ensure the architecture can be easily translated into a diagram.
- Avoid technical implementation details; focus on architectural design.

**Example:**

---

**1. High-Level Architecture Overview**

The proposed architecture is a scalable web application hosted on AWS,
following a multi-tier design that separates concerns across different
layers. The architecture comprises the following CDK stacks:

- **Networking Stack**
- **Database Stack**
- **Application Stack**
- **Authentication Stack**
- **Storage Stack**

**Interactions:**

- The **Networking Stack** provides the foundational network
infrastructure used by all other stacks.
- The **Application Stack** hosts the backend services and interfaces
with the **Database Stack** for data persistence.
- The **Authentication Stack** manages user authentication and interacts
with the **Application Stack** to secure API endpoints.
- The **Storage Stack** handles static assets and is accessed by the
**Application Stack** and frontend applications.

**Justification:**

This architecture promotes separation of concerns, enhances scalability,
and improves manageability by logically grouping resources. It leverages
AWS managed services to reduce operational overhead and ensures that each
component can scale independently.
```

```
---

**2. Detailed CDK Stack Descriptions**

**Networking Stack**

- **Purpose:** Establishes the network infrastructure for the
application.
- **AWS Resources:**
  - **Amazon VPC:** Provides isolated network space.
  - **Public and Private Subnets:** Segregate resources based on
  accessibility.
  - **Internet Gateway and NAT Gateways:** Enable internet access for
  public and private resources.
  - **Security Groups and Network ACLs:** Control inbound and outbound
  traffic.
- **Interactions:**
  - Used by all other stacks for deploying resources within the VPC.
- **Rationale:**
  - A dedicated networking stack allows centralized management of network
  resources and enhances security.

**Database Stack**

- **Purpose:** Manages data persistence for the application.
- **AWS Resources:**
  - **Amazon RDS (PostgreSQL):** Relational database for storing
  application data.
  - **DB Subnet Group:** Uses private subnets from the Networking Stack.
  - **Security Group:** Allows database access only from the Application
  Stack.
- **Interactions:**
  - The Application Stack connects to the database for CRUD operations.
- **Rationale:**
  - Amazon RDS provides a scalable and managed database solution,
  reducing administrative overhead.

**Application Stack**
```

```
- **Purpose:** Hosts the backend API and application logic.
- **AWS Resources:**
  - **AWS Lambda Functions:** Serverless compute for handling API
  requests.
  - **Amazon API Gateway:** Exposes RESTful API endpoints.
  - **AWS AppSync (optional):** For GraphQL APIs if required.
  - **IAM Roles:** Define permissions for accessing other AWS services.
- **Interactions:**
  - Communicates with the Database Stack for data operations.
  - Validates authentication tokens from the Authentication Stack.
- **Rationale:**
  - Serverless services offer scalability and cost-efficiency for
  variable workloads.

**Authentication Stack**

- **Purpose:** Manages user authentication and authorization.
- **AWS Resources:**
  - **Amazon Cognito User Pools:** Handles user sign-up, sign-in, and
  authentication.
  - **Amazon Cognito Identity Pools:** Provides temporary AWS credentials
  for accessing resources.
- **Interactions:**
  - The Application Stack uses Cognito for securing API endpoints.
- **Rationale:**
  - Cognito simplifies user management and integrates seamlessly with
  other AWS services.

**Storage Stack**

- **Purpose:** Stores static assets and handles file uploads/downloads.
- **AWS Resources:**
  - **Amazon S3 Buckets:** Store static content and user-uploaded files.
  - **Amazon CloudFront:** Distributes content globally with low latency.
- **Interactions:**
  - The Application Stack accesses S3 for storing and retrieving files.
  - Frontend applications fetch static assets via CloudFront.
- **Rationale:**
```

```
  - S3 and CloudFront provide a reliable and scalable solution for
  content storage and delivery.

---

**3. Assumptions and Considerations**

- **Assumptions:**
  - The application requires user authentication and data persistence.
  - Workloads are expected to be variable, benefiting from serverless
  solutions.
- **Constraints:**
  - Must use AWS services exclusively.
  - Needs to be scalable to handle increasing user demand.
- **Scalability and Security:**
  - AWS managed services like Lambda and RDS offer automatic scaling.
  - Security is enforced through VPC configurations, security groups, and
  IAM roles.
```

APPENDIX E – Custom development plan generation prompt

```
You are working in a software development agency and a project manager
and software architect approach you telling you that you're assigned to
work on a new project.
You are working on a fully AWS system that will be implemented using AWS
CDK with python, and you need to create a detailed development plan so
that developers can start developing the app.
You will receive the project main description, the project specification
and the selected architecture to this project.

**IMPORTANT**
The architecture is divided into CDK stacks that will be user to build
this application and it is the granularity you must use to create the
development plan.

----- MAIN DESCRIPTION -----
{{ main_description }}
----- END MAIN DESCRIPTION -----
----- PROJECT SPECIFICATIONS -----
{{ project_specs }}
----- END PROJECT SPECIFICATIONS -----
----- ARCHITECTURE -----
{{ architecture }}
----- END ARCHITECTURE -----
----- USE CASES -----
{{ use_cases }}
----- END USE CASES -----


Before we go into the coding part, I want you to split the development of
each CDK stack into smaller tasks so that it is easier to develop, debug
and make the application work.
```

```
Each task needs to be related only to the development of this application
and nothing else - once the application is fully working, that is it.
There shouldn't be a task for researching, deployment, writing
documentation, testing or anything that is not writing the actual code.

**IMPORTANT**
As an experienced tech lead you always follow rules on how to create
tasks. Dividing project into tasks is extremely important job and you
have to do it very carefully.

Now, based on the project details provided, think task by task and create
the entire development plan. Start from the project setup and specify
each task until the moment when the entire app should be fully working
while strictly following these rules:

Here are the revised rules tailored for a tech lead creating a
development plan for a Python AWS CDK (Cloud Development Kit)
application:

**Rule #1**
There should never be a task that is solely for testing or ensuring
something works; every task must involve coding. This should be
considered for every task, especially the final task of the project.
Testing the application will be done as part of each task.

**Rule #2**
This rule applies to the complexity of tasks.
You must ensure the project is not divided into tasks that are too small
or simple without reason, nor too large or complex, making them difficult
to develop, debug, and review. The project will already have the
workspace folder created and basic system dependencies installed, so
tasks for those are unnecessary.
Here are examples of poorly created tasks:

**too simple tasks**
- Set up a Python project and install the AWS CDK.
- Establish an S3 bucket using the default configuration.

**too complex tasks**
```

- Set up a Python project with an AWS CDK application that includes creating a VPC, setting up EC2 instances with specific security groups, creating Lambda functions, integrating with DynamoDB, and implementing a complete CI/CD pipeline.

Avoid creating tasks that are too simple or too complex. Aim for medium complexity tasks. Here are examples of good tasks:

good tasks
- Create a basic CDK stack that provisions a private S3 bucket.
- Implement a Lambda function using the AWS CDK. The Lambda should be triggered by an S3 bucket event and log the event details.

Rule #3
This rule applies to the number of tasks you will create.
Every application should have a different number of tasks depending on its complexity. Think task by task and create the minimum number of tasks that are relevant for this specific application. Here are some examples of apps with different complexity that can give you guidance on how many tasks you should create:

Example #1:
App description: "I want to create a CDK app that provisions an S3 bucket and a Lambda function that logs events."
Number of tasks: 2-3

Example #2:
App description: "Create a CDK app that sets up an API Gateway with Lambda integration, allowing users to make HTTP requests which trigger the Lambda."
Number of tasks: 3-5

Example #3:
App description: "A comprehensive CDK application that provisions a VPC, multiple EC2 instances, a load balancer, and integrates with an RDS database and S3 for storage."
Number of tasks: 6-12

Rule #4

```
This rule applies to writing task 'description'.
Every task must have a clear and very detailed description (minimum of 4
sentences, but can be more). It must be very clear so that even
developers who just moved to this project can execute them without
additional questions. It is not enough to just write something like
"Create an S3 bucket." You have to describe what needs to be done in
detail. Provide as many details as possible and ensure no information is
missing that could be needed for the task.
Here is an example of good and bad task descriptions:

**bad task**
Create a CDK stack for an S3 bucket

**good task**
In 'cdk_stack.py', create a new CDK stack class 'MyS3Stack'. In this
stack, define an S3 bucket with versioning enabled. The bucket should
have a lifecycle rule to transition objects to Glacier after 30 days. Add
the necessary IAM permissions to allow read/write access from a specific
Lambda function.

**Rule #5**
When creating and naming new files, ensure the file naming (camelCase,
kebab-case, underscore_case, etc.) is consistent with the best practices
and coding style of the language.
```

APPENDIX F – Requirements elicitation result

Chat Application - Requirements Specification

1. Introduction

A web-based chat application that allows users to communicate with each other in real-time. The application enables users to create accounts, log in with a username, search for other users by their username, and engage in one-on-one chats.

2. User Roles

- **Guest:** An unregistered visitor who can view information about the application but cannot access chat functionalities.
- **Registered User:** A user who has created an account and can use all chat features, including searching for other users and initiating chats.
- **Administrator:** Oversees the platform, manages user accounts, and ensures the application's smooth operation.

3. Functional Requirements

3.1 User Registration and Authentication

- **Account Creation:**
    - Users can sign up by providing a unique username and password.
    - Option to add an email address for account verification and password recovery.
- **Login and Logout:**
    - Secure login functionality using username and password.
    - Option to stay logged in using a "Remember Me" feature.
    - Users can securely log out from their accounts.
- **Password Management:**
    - Users can reset their passwords via email verification.
    - Password strength requirements to ensure account security.

```
**3.2 User Profile**

- **Profile Information:**
    - Users can view and edit their profile information, including:
        - Display name
        - Profile picture/avatar
        - Status message (e.g., "Online," "Away," custom messages)
- **Privacy Settings:**
    - Users can set their profiles to be searchable or private.
    - Option to block or unblock other users.

**3.3 User Search**

- **Search Functionality:**
    - Users can search for other registered users by entering their
    username.
    - Display search results with relevant user information (e.g.,
    display name, avatar).
- **Search Filters:**
    - Option to filter search results based on online status or other
    criteria.

**3.4 Chat Functionality**

- **One-on-One Chat:**
    - Users can initiate a private chat with another user from the search
    results or their contact list.
    - Real-time messaging with instant delivery of messages.
- **Chat Interface:**
    - User-friendly chat window displaying the conversation history.
    - Timestamp for each message.
    - Option to send text messages and basic emojis.
- **Chat History:**
    - Users can view past conversations.
    - Search within chat history for specific messages or keywords.

**3.5 Notifications**
```

```
- **Real-Time Notifications:**
    - Notification alerts for incoming messages when the user is online.
- **Offline Notifications:**
    - Email notifications for unread messages received while the user was
    offline (optional based on user settings).
- **Status Alerts:**
    - Notifications when a user comes online or goes offline.

**3.6 Administrative Functions**

- **User Management:**
    - Administrators can view, edit, or deactivate user accounts.
    - Ability to reset user passwords if necessary.
- **Content Moderation:**
    - Monitor and manage inappropriate content or behavior within chats.
    - Option to issue warnings or ban users violating terms of service.
- **Analytics:**
    - Access to usage statistics, such as active users, number of
    messages sent, etc.

**4. User Interface Requirements**

- **Responsive Design:**
    - The application should be accessible and optimized for desktop and
    mobile browsers.
- **Intuitive Navigation:**
    - Clear layout with easy access to search, chat windows, and user
    profiles.
- **User-Friendly Chat Interface:**
    - Clean and minimalistic design focusing on readability and ease of
    use.
- **Accessibility:**
    - Features to assist users with disabilities, such as keyboard
    navigation and screen reader compatibility.

**5. Non-Functional Requirements**

- **Performance:**
```

```
    - Fast loading times for all pages and swift message delivery to
    enhance user experience.
- **Scalability:**
    - Ability to accommodate an increasing number of users and message
    volumes without performance degradation.
- **Reliability:**
    - High availability with minimal downtime to ensure users can access
    the chat service whenever needed.
- **Security:**
    - Secure data transmission using SSL/TLS encryption.
    - Protection against common web vulnerabilities (e.g., SQL injection,
    XSS).
- **Data Privacy:**
    - Compliance with relevant data protection regulations (e.g., GDPR).
    - Secure storage of user data and chat histories.

**6. Assumptions**

- All users have access to a modern web browser and a stable internet
connection.
- The application will support text-based messaging; additional media
types (e.g., images, videos) may be considered in future iterations.
- Usernames are unique identifiers within the system.
- Users are responsible for maintaining the confidentiality of their
login credentials.

**7. Constraints**

- The application must be developed within a two-week development
timeframe, focusing on essential features for an MVP.
- Limited support for advanced features such as group chats, file
sharing, or voice/video calls in the initial version.
- User interface designs should adhere to basic accessibility standards
without extensive customization.
```

APPENDIX G – Architecture result

1. High-Level Architecture Overview

The proposed architecture is a scalable, secure, and real-time chat application hosted entirely on AWS. It leverages AWS managed services to ensure high availability, scalability, and minimal operational overhead. The architecture is organized into the following CDK stacks:

- **Networking Stack**
- **Authentication Stack**
- **Database Stack**
- **Backend Application Stack**
- **Real-Time Communication Stack**
- **Storage Stack**
- **Monitoring and Logging Stack**

Interactions:

- The **Networking Stack** establishes the foundational network infrastructure utilized by all other stacks.
- The **Authentication Stack** manages user authentication and interfaces with the **Backend Application Stack** to secure API endpoints.
- The **Database Stack** handles data persistence, including user profiles and chat histories.
- The **Backend Application Stack** hosts the application's business logic and APIs, communicating with both the **Database Stack** and **Real-Time Communication Stack**.
- The **Real-Time Communication Stack** manages real-time messaging functionalities.
- The **Storage Stack** handles static assets and any user-uploaded content.
- The **Monitoring and Logging Stack** oversees application performance, logging, and operational health.

Justification:

```
This architecture promotes a clear separation of concerns, enhances
scalability by utilizing serverless and managed services, and ensures
robust security. Logical grouping into CDK stacks facilitates organized
infrastructure management, ease of maintenance, and efficient resource
scaling.

---

**2. Detailed CDK Stack Descriptions**

---

**Networking Stack**

- **Purpose:**
    - Establishes the foundational network infrastructure for the
    application.

- **AWS Resources:**
    - **Amazon VPC:** Creates an isolated network environment.
    - **Public and Private Subnets:** Segregates resources based on
    accessibility requirements.
    - **Internet Gateway:** Provides internet access to public subnets.
    - **NAT Gateway:** Enables private subnets to access the internet
    securely.
    - **Security Groups and Network ACLs:** Controls inbound and outbound
    traffic to resources.

- **Interactions:**
    - All other stacks deploy their resources within the VPC provided by
    the Networking Stack.

- **Rationale:**
    - Centralizing network management enhances security and control over
    resource accessibility. Separating public and private subnets ensures
    that sensitive resources remain protected from direct internet
    exposure.

---
```

```
**Authentication Stack**

- **Purpose:**
    - Manages user authentication and authorization processes.

- **AWS Resources:**
    - **Amazon Cognito User Pools:** Handles user registration,
    authentication, and account management.
    - **Amazon Cognito Identity Pools:** Provides temporary AWS
    credentials for authenticated users.
    - **Lambda Triggers (optional):** Facilitates custom authentication
    flows or user attribute verification.

- **Interactions:**
    - The **Backend Application Stack** utilizes Cognito to secure API
    endpoints and manage user sessions.

- **Rationale:**
    - Amazon Cognito offers a scalable and secure user authentication
    service with built-in support for various authentication methods,
    reducing the need to manage custom authentication infrastructure.

---

**Database Stack**

- **Purpose:**
    - Manages data persistence for user profiles and chat histories.

- **AWS Resources:**
    - **Amazon DynamoDB:** A fully managed NoSQL database for storing
    user information and chat messages.
    - **DynamoDB Tables:**
        - **Users Table:** Stores user profiles and related data.
        - **Chats Table:** Stores chat sessions and messages.
    - **DynamoDB Streams (optional):** Enables real-time data processing
    or triggering events based on data changes.
```

```
- **Interactions:**
    - The **Backend Application Stack** interacts with DynamoDB to
    perform CRUD operations on user data and chat messages.

- **Rationale:**
    - DynamoDB provides low-latency, high-throughput data storage, making
    it ideal for real-time applications like chat apps. Its scalability
    ensures that the database can handle varying loads efficiently.

---

**Backend Application Stack**

- **Purpose:**
    - Hosts the application's backend APIs and business logic.

- **AWS Resources:**
    - **AWS Lambda Functions:** Implements serverless functions to handle
    API requests.
    - **Amazon API Gateway:** Exposes RESTful API endpoints for frontend
    communication.
    - **AWS AppSync (optional):** Manages GraphQL APIs if required.
    - **IAM Roles and Policies:** Defines permissions for accessing other
    AWS services securely.

- **Interactions:**
    - Communicates with the **Database Stack** to access and manipulate
    user and chat data.
    - Integrates with the **Authentication Stack** to verify user
    identities and secure endpoints.

- **Rationale:**
    - Serverless architecture using Lambda and API Gateway offers
    scalability, cost-efficiency, and reduced operational management. It
    allows the application to handle varying traffic loads seamlessly.

---

**Real-Time Communication Stack**
```

```
- **Purpose:**
    - Manages real-time messaging capabilities for the chat application.

- **AWS Resources:**
    - **Amazon API Gateway WebSocket APIs:** Facilitates real-time,
    two-way communication between clients and the backend.
    - **AWS Lambda Functions:** Handles WebSocket connection events and
    message routing.
    - **Amazon DynamoDB or Amazon ElastiCache (Redis):** Manages session
    state and message delivery if required.

- **Interactions:**
    - Integrates with the **Backend Application Stack** to process and
    route messages in real time.

- **Rationale:**
    - WebSocket APIs in API Gateway provide a scalable solution for
    real-time communication without the need to manage persistent server
    connections manually.

---

**Storage Stack**

- **Purpose:**
    - Stores static assets, user-uploaded files, and any media related to
    chats.

- **AWS Resources:**
    - **Amazon S3 Buckets:** Stores static website content and
    user-uploaded files such as profile pictures.
    - **Amazon CloudFront:** Distributes content globally with low
    latency.
    - **AWS Lambda@Edge (optional):** Facilitates custom content
    processing at CloudFront edge locations.

- **Interactions:**
```

```
    - The **Backend Application Stack** accesses S3 for storing and
    retrieving user-uploaded content.
    - Frontend applications fetch static assets via CloudFront for
    improved performance.

- **Rationale:**
    - Amazon S3 provides durable and scalable storage for static content,
    while CloudFront enhances content delivery speeds globally, improving
    user experience.

---

**Monitoring and Logging Stack**

- **Purpose:**
    - Oversees application performance, monitoring, and logging for
    maintenance and troubleshooting.

- **AWS Resources:**
    - **Amazon CloudWatch:** Collects and monitors logs, metrics, and
    sets up alarms.
    - **AWS X-Ray (optional):** Facilitates tracing of requests and
    analyzing performance bottlenecks.
    - **AWS CloudTrail:** Logs API calls for auditing and compliance
    purposes.

- **Interactions:**
    - Aggregates logs and metrics from all other stacks, enabling
    centralized monitoring and alerting.

- **Rationale:**
    - Effective monitoring and logging are critical for maintaining
    application health, troubleshooting issues, and ensuring security
    compliance. Leveraging CloudWatch and CloudTrail provides
    comprehensive visibility into system operations.

---

**3. Assumptions and Considerations**
```

```
- **Assumptions:**
    - The chat application requires scalability to handle a growing user
    base and message volume.
    - Real-time communication is essential for user engagement.
    - Users will access the application via modern web browsers with
    stable internet connections.
    - The application will focus on one-on-one chats in the MVP phase,
    with potential for future enhancements like group chats.
    - Usernames are unique identifiers within the system to facilitate
    accurate user searches and chat initiation.

- **Constraints:**
    - The architecture must be entirely hosted on AWS, utilizing only AWS
    services.
    - Development must prioritize essential features to align with the
    MVP focus and a manageable development timeframe.

- **Scalability and Security:**
    - **Scalability:**
        - Serverless services like AWS Lambda and Amazon API Gateway
        automatically scale with demand.
        - DynamoDB scales seamlessly to handle increasing data volumes
        and request rates.
        - Amazon CloudFront ensures scalable content delivery without
        degradation in performance.
    - **Security:**
        - User data is secured through Amazon Cognito and IAM roles,
        ensuring proper authentication and authorization.
        - Data transmission is protected via SSL/TLS encryption.
        - Amazon VPC and security groups restrict access to sensitive
        resources.
        - Compliance with data protection regulations (e.g., GDPR) is
        maintained by leveraging AWS's compliant services and
        configurations.

---
```

```
This architectural design leverages AWS's robust suite of managed
services to deliver a secure, scalable, and efficient chat application.
By organizing resources into logical CDK stacks, the architecture ensures
maintainability and facilitates future enhancements.
```

APPENDIX H – End-to-end use case result

```
**3. Interaction of Main Use Cases with AWS Services**

This section outlines how the primary user interactions with the chat
application map to the AWS services defined in the architecture. It
details data handling and the end-to-end workflow for each use case:
**User Login**, **User Search**, and **Starting a Chat**.

---

### **3.1 User Login**

**Use Case Overview:**

A registered user logs into the chat application using their username and
password to access chat functionalities.

**Interaction with AWS Services:**

1. **Frontend Application:**

   - **User Input:** The user enters their username and password on the
   login interface.
   - **API Request:** The frontend sends an authentication request to the
   backend via **Amazon API Gateway**.

2. **Backend Application Stack:**

   - **Amazon API Gateway:**
     - **Endpoint:** Receives the login request at a designated `/login`
     endpoint.
     - **Integration:** Triggers an **AWS Lambda** function to handle
     authentication.

   - **AWS Lambda Function:**
```

```
     - **Authentication Logic:** Utilizes **Amazon Cognito User Pools**
     to authenticate the user credentials.
     - **Token Generation:** Upon successful authentication, Cognito
     generates authentication tokens (ID Token, Access Token, Refresh
     Token).

3. **Authentication Stack:**

   - **Amazon Cognito User Pools:**
     - **User Verification:** Validates the provided username and
     password.
     - **Token Management:** Issues tokens that encapsulate user identity
     and session information.

4. **Frontend Application:**

   - **Token Handling:** Receives and securely stores the authentication
   tokens (e.g., in local storage or secure cookies).
   - **Session Management:** Uses tokens to authenticate subsequent API
   requests and maintain user sessions.

**Data Handling:**

- **Secure Transmission:** All authentication data is transmitted over
HTTPS to ensure data privacy and integrity.
- **Credential Storage:** User credentials are securely managed by Amazon
Cognito, adhering to best security practices.
- **Token Security:** Authentication tokens are stored securely on the
client side to prevent unauthorized access.

**End-to-End Workflow:**

1. **Login Initiation:**
   - User enters username and password on the login page.

2. **Authentication Request:**
   - Frontend sends a POST request to the `/login` endpoint via Amazon
   API Gateway, including the entered credentials.
```

```
3. **Credential Verification:**
   - API Gateway invokes the associated Lambda function.
   - The Lambda function communicates with Amazon Cognito User Pools to
   validate credentials.

4. **Token Issuance:**
   - Upon successful validation, Cognito generates authentication tokens
   and returns them to the Lambda function.

5. **Token Delivery:**
   - Lambda function sends the tokens back to the frontend through API
   Gateway.

6. **Session Establishment:**
   - Frontend stores the tokens and marks the user as authenticated,
   allowing access to chat functionalities.

---

### **3.2 User Search**

**Use Case Overview:**

A registered user searches for other users by their username to initiate
a chat.

**Interaction with AWS Services:**

1. **Frontend Application:**

   - **Search Input:** User enters the target username in the search bar.
   - **API Request:** Frontend sends a search query to the backend via
   **Amazon API Gateway**.

2. **Backend Application Stack:**

   - **Amazon API Gateway:**
     - **Endpoint:** Receives the search request at a designated
     `/search` endpoint.
```

```
     - **Integration:** Triggers an **AWS Lambda** function to handle the
     search logic.

   - **AWS Lambda Function:**
     - **Database Query:** Queries the **Users Table** in **Amazon
     DynamoDB** to find matching usernames.
     - **Data Processing:** Formats the retrieved user data for frontend
     consumption.

3. **Database Stack:**

   - **Amazon DynamoDB - Users Table:**
     - **Data Schema:** Stores user profiles, including usernames,
     display names, avatars, and privacy settings.
     - **Indexing:** Utilizes primary and secondary indexes to facilitate
     efficient search operations.

4. **Backend Application Stack:**

   - **Response Formation:** Lambda function aggregates and structures
   the search results.
   - **API Response:** Sends the formatted data back to the frontend via
   API Gateway.

5. **Frontend Application:**

   - **Display Results:** Receives and displays the search results,
   showcasing relevant user information.
   - **Interaction Options:** Provides options to view user profiles or
   initiate chats based on search results.

**Data Handling:**

- **Search Queries:** Transmitted securely over HTTPS from frontend to
API Gateway.
- **User Data Retrieval:** DynamoDB efficiently retrieves user
information based on the search criteria.
- **Data Privacy:** Ensures that only authorized and appropriate user
data is displayed, respecting privacy settings.
```

End-to-End Workflow:

1. **Search Initiation:**
   - Authenticated user inputs a username in the search interface.

2. **Search Request:**
   - Frontend sends a GET or POST request to the `/search` endpoint via API Gateway, including the search term.

3. **Database Query:**
   - API Gateway invokes the Lambda function responsible for handling search requests.
   - Lambda function queries the Users Table in DynamoDB using the provided username.

4. **Result Processing:**
   - DynamoDB returns matching user profiles to the Lambda function.
   - Lambda function processes and formats the data appropriately.

5. **Response Delivery:**
   - Lambda function sends the search results back to the frontend through API Gateway.

6. **Result Presentation:**
   - Frontend displays the search results, allowing the user to select a profile for further interaction.

3.3 Starting a Chat

Use Case Overview:

A registered user initiates a one-on-one real-time chat with another user found through the search functionality.

Interaction with AWS Services:

```
1. **Frontend Application:**

   - **Chat Initiation:** User selects another user from the search
   results and clicks to start a chat.
   - **WebSocket Connection:** Frontend establishes a WebSocket
   connection to the backend via **Amazon API Gateway WebSocket API**.

2. **Real-Time Communication Stack:**

   - **Amazon API Gateway WebSocket API:**
     - **Connection Management:** Handles the establishment and
     maintenance of WebSocket connections.
     - **Message Routing:** Routes incoming messages to appropriate
     Lambda functions based on message type.

   - **AWS Lambda Functions:**
     - **Connection Handler:** Manages connection and disconnection
     events.
     - **Message Processor:** Processes incoming messages and routes them
     to recipients.

3. **Backend Application Stack:**

   - **AWS Lambda Function:**
     - **Session Management:** Associates WebSocket connection IDs with
     user identities in **DynamoDB**.
     - **Message Handling:** Receives messages from senders and forwards
     them to recipients.

4. **Database Stack:**

   - **Amazon DynamoDB - Chats Table:**
     - **Data Schema:** Stores chat sessions and messages, including
     sender ID, recipient ID, timestamp, and message content.
     - **Scalability:** Handles high volumes of chat data with low
     latency.

5. **Frontend Application:**
```

- **Real-Time Messaging:** Receives incoming messages through the established WebSocket connection.
- **Chat Interface:** Displays messages in real-time, updating the conversation view dynamically.

Data Handling:

- **Real-Time Data Transmission:** Utilizes WebSocket connections for bidirectional, low-latency communication.
- **Message Storage:** All chat messages are persistently stored in DynamoDB for history retrieval and auditing.
- **Connection Management:** WebSocket connection IDs are securely managed and mapped to user identities to ensure message integrity.

End-to-End Workflow:

1. **Chat Initiation:**
   - User selects another user from the search results and opts to start a chat.

2. **WebSocket Connection:**
   - Frontend initiates a WebSocket connection to the API Gateway WebSocket API.
   - API Gateway triggers a Lambda function to handle the connection event, associating the connection ID with the user in DynamoDB.

3. **Real-Time Communication Establishment:**
   - Both users have active WebSocket connections managed by API Gateway.
   - Frontend applications are now capable of sending and receiving messages in real-time.

4. **Message Sending:**
   - User types a message and sends it through the chat interface.
   - Message is transmitted via the WebSocket connection to API Gateway.

5. **Message Processing:**
   - API Gateway routes the incoming message to the appropriate Lambda function.

```
   - Lambda function processes the message, stores it in the Chats Table
   in DynamoDB, and retrieves the recipient's connection ID.

6. **Message Delivery:**
   - Lambda function sends the message to the recipient's WebSocket
   connection through API Gateway.
   - Recipient's frontend receives the message instantly and displays it
   in the chat interface.

7. **Chat History Access:**
   - Users can scroll through past messages, which are fetched from
   DynamoDB via API calls handled by Lambda functions.

---

### **3.4 Data Handling and Management**

**Data Storage:**

- **Amazon DynamoDB:**
  - **Users Table:** Stores user profiles, ensuring quick retrieval
  during search operations.
  - **Chats Table:** Maintains chat histories, enabling users to access
  past conversations seamlessly.

**Data Security:**

- **Encryption:**
  - **At Rest:** Data stored in DynamoDB is automatically encrypted using
  AWS-managed keys.
  - **In Transit:** All data transmissions occur over HTTPS and WSS (for
  WebSocket) protocols, ensuring end-to-end encryption.

- **Access Control:**
  - **IAM Roles and Policies:** Define granular permissions for Lambda
  functions to access only necessary resources.
  - **Amazon Cognito:** Manages user authentication and authorization,
  enforcing access controls based on user roles.
```

```
**Data Flow:**

1. **User Data Management:**
   - User registration and profile updates are handled by Lambda
   functions interacting with Cognito and the Users Table in DynamoDB.

2. **Chat Data Management:**
   - Real-time messages are transmitted via WebSocket connections,
   processed by Lambda functions, and stored in the Chats Table.
   - Chat histories are retrieved through API calls that query DynamoDB,
   ensuring efficient data access.

3. **Session Management:**
   - WebSocket connection IDs are mapped to user identities within
   DynamoDB, facilitating accurate message routing and session tracking.

**Data Privacy:**

- **Compliance:**
  - Ensures adherence to data protection regulations (e.g., GDPR) by
  leveraging AWS's compliance certifications and best practices.

- **User Control:**
  - Users have control over their data, including the ability to manage
  privacy settings and access their chat histories.

---

### **3.5 End-to-End Workflow**

This section describes the comprehensive journey of a user interacting
with the chat application, from authentication to real-time messaging.

**Example Workflow: Authentication and Chat**

1. **User Registration (Assumed):**

   - **Registration Process:**
```

```
    - User signs up using the frontend application by providing a unique
    username and password.
    - Frontend sends a registration request to the `/register` endpoint
    via API Gateway.
    - Lambda function processes the registration, creating a new user in
    **Amazon Cognito User Pools**.
    - User receives a confirmation email for account verification.

2. **User Login:**

   - **Authentication:**
     - User enters their username and password on the login page.
     - Frontend sends a POST request to the `/login` endpoint via API
     Gateway with the credentials.
     - Lambda function authenticates the user through Cognito.
     - Upon successful authentication, Cognito issues authentication
     tokens.
     - Frontend stores the tokens and marks the user as authenticated.

3. **User Search:**

   - **Initiating a Search:**
     - Authenticated user enters a target username in the search bar.
     - Frontend sends a search request to the `/search` endpoint via API
     Gateway, including the search term.
     - Lambda function queries the **Users Table** in DynamoDB for
     matching usernames.
     - Search results are returned to the frontend, displaying relevant
     user profiles.

4. **Initiating a Chat:**

   - **Starting a Conversation:**
     - User selects a profile from the search results and opts to start a
     chat.
     - Frontend establishes a WebSocket connection to the **Amazon API
     Gateway WebSocket API**.
     - API Gateway triggers a Lambda function to handle the connection
     event, associating the connection ID with the user in DynamoDB.
```

5. **Real-Time Messaging:**

   - **Sending a Message:**
     - User types a message and sends it through the chat interface.
     - Message is transmitted via the WebSocket connection to API Gateway.
     - API Gateway routes the message to the corresponding Lambda function.
     - Lambda function processes the message, storing it in the **Chats Table** in DynamoDB.
     - Lambda function retrieves the recipient's connection ID from DynamoDB and sends the message through API Gateway to the recipient's WebSocket connection.

   - **Receiving a Message:**
     - Recipient's frontend receives the message in real-time via the WebSocket connection.
     - Message is displayed in the chat interface, ensuring instant communication.

6. **Accessing Chat History:**

   - **Retrieving Past Messages:**
     - User scrolls through the chat interface to view previous messages.
     - Frontend sends a request to the `/chat/history` endpoint via API Gateway.
     - Lambda function queries the **Chats Table** in DynamoDB for past messages.
     - Retrieved chat history is sent back to the frontend for display.

7. **Session Termination:**

   - **Closing the Chat:**
     - User ends the chat session or disconnects.
     - Frontend closes the WebSocket connection.
     - API Gateway triggers a Lambda function to handle the disconnection event, updating the session state in DynamoDB.

```
**Visualization of the Workflow:**

1. **User Interaction:**
   - Registration/Login → Search Users → Initiate Chat → Send/Receive
   Messages → Access Chat History

2. **AWS Service Interaction:**
   - Frontend  API Gateway  Lambda Functions  DynamoDB  Cognito
   WebSocket API

3. **Data Flow:**
   - User Credentials → Authentication Tokens → User Profiles → Chat
   Messages → Session Data

---

### **3.6 Summary of Data Handling**

- **Authentication Data:**
  - Managed by Amazon Cognito, ensuring secure handling of user
  credentials and session tokens.

- **User Profiles:**
  - Stored and retrieved from the Users Table in DynamoDB, enabling
  efficient search and profile management.

- **Chat Messages:**
  - Transmitted in real-time via WebSocket connections, processed by
  Lambda functions, and persistently stored in the Chats Table in
  DynamoDB.

- **Session Management:**
  - Connection IDs and user mappings are maintained in DynamoDB,
  facilitating accurate message routing and session tracking.

- **Data Security and Compliance:**
  - All data interactions are secured through encryption and controlled
  access, adhering to data protection regulations and best practices.
```

3.7 Next Steps for End-to-End Workflow Implementation

To ensure a seamless end-to-end workflow, the following steps should be undertaken:

1. **Frontend Development:**

   - **Authentication Interface:** Implement user registration and login forms, handling token storage and session management.

   - **Search Functionality:** Develop the search bar interface, integrating API calls to fetch and display user profiles.

   - **Chat Interface:** Design the real-time chat interface, establishing WebSocket connections and handling message display.

2. **Backend Development:**

   - **Lambda Functions:** Develop and deploy Lambda functions for authentication, user search, chat initiation, message processing, and session management.

   - **API Gateway Configuration:** Set up RESTful and WebSocket APIs, defining routes, integrations, and security settings.

3. **Database Setup:**

   - **DynamoDB Tables:** Define and configure the Users and Chats tables with appropriate schemas and indexing for efficient data retrieval.

4. **Authentication Integration:**

   - **Cognito Configuration:** Set up User Pools and Identity Pools, defining authentication flows and user attributes.

5. **Real-Time Communication Setup:**

```
   - **WebSocket API Configuration:** Establish WebSocket routes and
   integrate them with corresponding Lambda functions for message
   handling.

6. **Security Implementations:**

   - **IAM Roles and Policies:** Define strict access controls for Lambda
   functions and other AWS resources to ensure least privilege.

   - **SSL/TLS Configuration:** Ensure all API endpoints and WebSocket
   connections are secured with SSL/TLS certificates.

7. **Testing and Validation:**

   - **Unit and Integration Testing:** Verify the functionality of each
   component and their interactions.

   - **End-to-End Testing:** Simulate user workflows to ensure seamless
   operation across all use cases.

8. **Monitoring and Logging:**

   - **CloudWatch Setup:** Implement monitoring and logging for all
   Lambda functions, API Gateway, and other services to track performance
   and troubleshoot issues.

9. **Deployment Automation:**

   - **CDK Deployment:** Utilize AWS CDK with Python to define and deploy
   infrastructure as code, ensuring reproducibility and scalability.

10. **Documentation and Training:**

    - **User Guides:** Create documentation for end-users detailing how
    to use the chat application.

    - **Developer Documentation:** Document the architecture, codebase,
    and deployment processes for ongoing maintenance and future
    development.
```

This comprehensive mapping ensures that the chat application's main use cases are effectively supported by the chosen AWS services. By following the outlined interactions and workflows, the development team can build a secure, scalable, and user-friendly chat application that aligns with the project's requirements.

APPENDIX I – End-to-end use case diagrams result

````
**Use Case: User Registration and Authentication**

```mermaid
sequenceDiagram
    participant User
    participant Frontend
    participant API_Gateway
    participant LambdaAuth
    participant Cognito
    participant DynamoDB

    User->>Frontend: Submit Sign Up/Login Request
    Frontend->>API_Gateway: Send Authentication Request
    API_Gateway->>LambdaAuth: Trigger Authentication Lambda
    LambdaAuth->>Cognito: Register/Authenticate User
    Cognito-->>LambdaAuth: Authentication Response
    LambdaAuth->>DynamoDB: Store/Retrieve User Data
    LambdaAuth-->>API_Gateway: Return Token/Status
    API_Gateway-->>Frontend: Authentication Response
    Frontend-->>User: Display Success/Failure Message
```

---

**Use Case: Search User**

```mermaid
sequenceDiagram
    participant User
    participant Frontend
    participant API_Gateway
    participant LambdaSearch
    participant DynamoDB
````
````

```
    User->>Frontend: Enter Username to Search
    Frontend->>API_Gateway: Send Search Request
    API_Gateway->>LambdaSearch: Trigger Search Lambda
    LambdaSearch->>DynamoDB: Query Users Table
    DynamoDB-->>LambdaSearch: Return User Data
    LambdaSearch-->>API_Gateway: Search Results
    API_Gateway-->>Frontend: Return Search Results
    Frontend-->>User: Display User Profiles
```

Use Case: Start Chat

```mermaid
sequenceDiagram
    participant User_A
    participant Frontend
    participant API_Gateway
    participant LambdaChat
    participant DynamoDB
    participant APIGW_WebSocket
    participant LambdaRealTime

    User_A->>Frontend: Select User to Chat
    Frontend->>API_Gateway: Initiate Chat Request
    API_Gateway->>LambdaChat: Trigger Chat Lambda
    LambdaChat->>DynamoDB: Create/Retrieve Chat Session
    DynamoDB-->>LambdaChat: Chat Session Data
    LambdaChat-->>API_Gateway: Return Chat Session Info
    API_Gateway-->>Frontend: Provide Chat Session Details
    Frontend->>APIGW_WebSocket: Open WebSocket Connection
    APIGW_WebSocket-->>LambdaRealTime: Manage Real-Time Messaging
    LambdaRealTime->>DynamoDB: Store Messages
    LambdaRealTime->>APIGW_WebSocket: Deliver Messages to Recipient
    User_B->>Frontend: Receive Incoming Message
```

```
---

**Use Case: Send and Receive Messages**

```mermaid
sequenceDiagram
    participant User_A
    participant Frontend
    participant APIGW_WebSocket
    participant LambdaRealTime
    participant DynamoDB
    participant User_B

    User_A->>Frontend: Send Message
    Frontend->>APIGW_WebSocket: Transmit Message via WebSocket
    APIGW_WebSocket->>LambdaRealTime: Forward Message
    LambdaRealTime->>DynamoDB: Store Message in Chats Table
    LambdaRealTime-->>APIGW_WebSocket: Acknowledge Message Receipt
    APIGW_WebSocket-->>Frontend: Confirm Message Sent
    Frontend-->>User_A: Display Sent Message
    LambdaRealTime->>APIGW_WebSocket: Push Message to Recipient
    APIGW_WebSocket-->>Frontend (User_B): Deliver Incoming Message
    Frontend-->>User_B: Display Received Message
```

Use Case: Receive Notifications

```mermaid
sequenceDiagram
    participant User
    participant Frontend
    participant API_Gateway
    participant LambdaNotifications
    participant SNS

    User->>Frontend: Perform Action (e.g., Receive New Message)
    LambdaNotifications->>SNS: Publish Notification
```
```

```
    SNS-->>User: Send Notification via Preferred Channel (e.g., Email)
    User->>Frontend: View Notification
```

These Mermaid diagrams illustrate the interaction between users and AWS services for each primary use case in the chat application. Each diagram outlines the step-by-step workflow, highlighting how data flows through various AWS components to achieve the desired functionality.

APPENDIX J – Overall architecture diagram result

```mermaid
graph TD
    %% Users
    User[User]

    %% Networking Stack
    subgraph Networking_Stack
        VPC[VPC]
        PublicSubnet[Public Subnet]
        PrivateSubnet[Private Subnet]
        InternetGateway[Internet Gateway]
        NATGateway[NAT Gateway]
        SecurityGroups[Security Groups]
    end

    %% Authentication Stack
    subgraph Authentication_Stack
        Cognito[Cognito User Pools]
    end

    %% Application Stack
    subgraph Application_Stack
        APIGateway[API Gateway]
        LambdaBackend[AWS Lambda]
        IAMRoles[IAM Roles]
    end

    %% Database Stack
    subgraph Database_Stack
        DynamoDB[DynamoDB]
    end

    %% Storage Stack
```

```
subgraph Storage_Stack
    S3[S3 Bucket]
end

%% Notification Stack
subgraph Notification_Stack
    SNS[Amazon SNS]
    SES[Amazon SES]
end

%% Search Stack
subgraph Search_Stack
    ElasticSearch[Amazon Elasticsearch]
end

%% Content Delivery Stack
subgraph Content_Delivery_Stack
    CloudFront[CloudFront]
end

%% Monitoring and Logging Stack
subgraph Monitoring_Logging_Stack
    CloudWatch[CloudWatch]
    CloudTrail[CloudTrail]
end

%% Interactions
User -->|Access| CloudFront
CloudFront --> S3
User -->|Authenticate| Cognito
Cognito --> APIGateway
User -->|API Request| APIGateway
APIGateway --> LambdaBackend
LambdaBackend --> DynamoDB
LambdaBackend --> S3
LambdaBackend --> ElasticSearch
LambdaBackend --> SNS
SNS --> SES
LambdaBackend -->|Logs| CloudWatch
```

```
    LambdaBackend -->|Audit| CloudTrail

    %% Security Interactions
    VPC --> PublicSubnet
    VPC --> PrivateSubnet
    PublicSubnet --> InternetGateway
    PrivateSubnet --> NATGateway
    APIGateway -->|Uses| IAMRoles
    LambdaBackend -->|Security Group| SecurityGroups
    DynamoDB -->|Security Group| SecurityGroups
    S3 -->|Security Group| SecurityGroups
    ElasticSearch -->|Security Group| SecurityGroups
    CloudFront -->|Distributes| S3
    User -->|Receives Notifications| SNS
    User -->|Receives Emails| SES
```

APPENDIX K – Development plan result

```
### **Development Plan for Chat Application Using AWS CDK with Python**

#### **1. Networking Stack**

**Task 1: Define the VPC with Public and Private Subnets**

- **Description:**
  - In the `networking_stack.py` file, create a new CDK stack class named
  `NetworkingStack`.
  - Within this stack, define an Amazon VPC with a CIDR block suitable
  for the application (e.g., `10.0.0.0/16`).
  - Configure the VPC to include both public and private subnets across
  at least two Availability Zones to ensure high availability.
  - Attach an Internet Gateway to the VPC for internet access in public
  subnets.
  - Set up NAT Gateways in public subnets to allow instances in private
  subnets to access the internet securely.
  - Implement necessary route tables and associations to ensure proper
  routing between subnets and gateways.

**Task 2: Configure Security Groups and Network ACLs**

- **Description:**
  - Within the `NetworkingStack`, define Security Groups for different
  resource types (e.g., Lambda functions, API Gateway).
  - Configure inbound and outbound rules to restrict access based on the
  application's security requirements.
  - Create Network ACLs to provide an additional layer of security at the
  subnet level.
  - Ensure that private subnets are not accessible directly from the
  internet, enhancing the security posture of the application.
  - Document all security group rules and network ACL configurations for
  clarity and future reference.

#### **2. Authentication Stack**
```

```
**Task 3: Set Up Amazon Cognito User Pools**

- **Description:**
  - In the `authentication_stack.py` file, create a new CDK stack class
  named `AuthenticationStack`.
  - Define an Amazon Cognito User Pool to manage user registration,
  authentication, and account management.
  - Configure the User Pool with attributes such as username, password
  policies, and email verification settings.
  - Enable Lambda triggers for custom authentication flows if necessary.
  - Ensure that the User Pool is integrated with the VPC defined in the
  Networking Stack for secure access.

**Task 4: Configure Cognito Identity Pools for AWS Credentials**

- **Description:**
  - Within the `AuthenticationStack`, set up an Amazon Cognito Identity
  Pool to provide temporary AWS credentials to authenticated users.
  - Link the Identity Pool with the previously created User Pool to
  enable seamless authentication.
  - Define IAM roles and policies that specify the permissions granted to
  authenticated users.
  - Ensure that the Identity Pool settings align with the application's
  security and access requirements.
  - Test the integration to verify that authenticated users receive
  appropriate AWS credentials.

#### **3. Database Stack**

**Task 5: Create DynamoDB Tables for Users and Chats**

- **Description:**
  - In the `database_stack.py` file, create a new CDK stack class named
  `DatabaseStack`.
  - Define two Amazon DynamoDB tables: `UsersTable` and `ChatsTable`.
    - **UsersTable:** Include attributes such as `username` (primary
    key), `displayName`, `avatarUrl`, `statusMessage`, and
    `privacySettings`.
```

```
    - **ChatsTable:** Include attributes like `chatId` (primary key),
    `senderId`, `recipientId`, `timestamp`, and `messageContent`.
  - Configure appropriate primary keys and secondary indexes to optimize
  query performance for user searches and chat retrievals.
  - Enable DynamoDB Streams on the ChatsTable if real-time processing of
  chat messages is required.
  - Set provisioned or on-demand capacity based on expected usage
  patterns to ensure scalability.

**Task 6: Implement Access Permissions for DynamoDB Tables**

- **Description:**
  - Within the `DatabaseStack`, define IAM roles and policies that grant
  the Backend Application Stack necessary permissions to interact with
  the DynamoDB tables.
  - Ensure that Lambda functions accessing DynamoDB have the least
  privilege required to perform their operations.
  - Attach policies that allow actions such as `GetItem`, `PutItem`,
  `Query`, and `Scan` on the UsersTable and ChatsTable as needed.
  - Verify that no overly permissive policies are granted, maintaining a
  strong security posture.
  - Document the IAM roles and their associated permissions for future
  maintenance and auditing.

#### **4. Backend Application Stack**

**Task 7: Develop Lambda Functions for Authentication and User
Management**

- **Description:**
  - In the `backend_application_stack.py` file, create a new CDK stack
  class named `BackendApplicationStack`.
  - Implement AWS Lambda functions to handle user registration, login,
  logout, and profile management.
    - **RegisterUserFunction:** Handles user sign-up requests,
    interacting with Amazon Cognito to create new users.
    - **LoginUserFunction:** Manages user authentication, validating
    credentials and issuing tokens via Cognito.
```

```
    - **LogoutUserFunction:** Facilitates user logout by revoking tokens
    or clearing sessions.
    - **ManageProfileFunction:** Allows users to view and update their
    profile information, interacting with the UsersTable in DynamoDB.
  - Ensure each Lambda function is written in Python, following best
  coding practices and standards.
  - Configure environment variables and necessary IAM permissions for
  each Lambda function to access Cognito and DynamoDB resources securely.

**Task 8: Set Up API Gateway Endpoints for Backend APIs**

- **Description:**
  - Within the `BackendApplicationStack`, configure Amazon API Gateway to
  expose RESTful endpoints for frontend interaction.
  - Define endpoints such as `/register`, `/login`, `/logout`,
  `/profile`, and `/search`.
  - Integrate each endpoint with the corresponding Lambda functions
  developed in Task 7.
  - Configure request and response mappings, ensuring that API Gateway
  correctly transforms data between the frontend and Lambda functions.
  - Enable CORS settings to allow the frontend application to communicate
  with the API securely.
  - Implement throttling and rate limiting to protect backend services
  from abuse and ensure fair usage.

#### **5. Real-Time Communication Stack**

**Task 9: Configure WebSocket API for Real-Time Messaging**

- **Description:**
  - In the `real_time_communication_stack.py` file, create a new CDK
  stack class named `RealTimeCommunicationStack`.
  - Set up an Amazon API Gateway WebSocket API to facilitate real-time,
  two-way communication between clients and the backend.
  - Define WebSocket routes such as `$connect`, `$disconnect`, and custom
  `$message` routes to handle different message types.
  - Integrate each route with corresponding Lambda functions responsible
  for managing connections and message routing.
```

```
  - Enable usage plans and throttling settings to ensure the WebSocket
  API can scale with user demand while maintaining performance.

**Task 10: Develop Lambda Functions for WebSocket Connection and
Messaging**

- **Description:**
  - Within the `RealTimeCommunicationStack`, implement AWS Lambda
  functions to handle WebSocket events.
    - **ConnectHandlerFunction:** Manages new WebSocket connections,
    storing connection IDs and associating them with user identities in
    DynamoDB.
    - **DisconnectHandlerFunction:** Handles disconnections, cleaning up
    session data and updating user status.
    - **MessageHandlerFunction:** Processes incoming messages, stores
    them in the ChatsTable, and forwards messages to the intended
    recipients via their connection IDs.
  - Ensure each Lambda function is written in Python, following best
  coding practices and standards.
  - Configure environment variables and necessary IAM permissions for
  each Lambda function to interact with DynamoDB and API Gateway
  securely.

#### **6. Storage Stack**

**Task 11: Set Up Amazon S3 Buckets for Static Assets and User Uploads**

- **Description:**
  - In the `storage_stack.py` file, create a new CDK stack class named
  `StorageStack`.
  - Define Amazon S3 buckets to store static website content (e.g., HTML,
  CSS, JavaScript files) and user-uploaded assets such as profile
  pictures.
    - **StaticAssetsBucket:** Stores frontend static files, configured
    for website hosting if necessary.
    - **UserUploadsBucket:** Stores user-generated content, with
    appropriate access controls to ensure privacy and security.
```

```
  - Implement bucket policies to restrict access, ensuring that only
  authorized resources (e.g., Lambda functions, CloudFront) can interact
  with the buckets.
  - Enable versioning and lifecycle policies on the buckets to manage
  storage costs and maintain data integrity.

**Task 12: Configure CloudFront Distribution for Content Delivery**

- **Description:**
  - Within the `StorageStack`, set up an Amazon CloudFront distribution
  to deliver static assets and user-uploaded content with low latency
  globally.
  - Configure CloudFront to use the StaticAssetsBucket and
  UserUploadsBucket as origin sources.
  - Implement caching policies to optimize content delivery performance.
  - Enable HTTPS support to ensure secure communication between clients
  and CloudFront.
  - Set up necessary invalidation rules to handle content updates and
  cache purging when static assets are modified.

#### **7. Monitoring and Logging Stack**

**Task 13: Implement CloudWatch Logging for All Lambda Functions**

- **Description:**
  - In the `monitoring_logging_stack.py` file, create a new CDK stack
  class named `MonitoringLoggingStack`.
  - Configure Amazon CloudWatch Logs to capture logs from all Lambda
  functions across the Backend Application Stack and Real-Time
  Communication Stack.
  - Define log groups and set appropriate retention policies to manage
  storage costs and comply with data retention requirements.
  - Ensure that all Lambda functions have the necessary permissions to
  write logs to CloudWatch.
  - Set up structured logging within Lambda functions to facilitate
  easier monitoring and troubleshooting.

**Task 14: Set Up CloudWatch Alarms for Critical Metrics**
```

- **Description:**
  - Within the `MonitoringLoggingStack`, configure Amazon CloudWatch Alarms to monitor critical application metrics such as Lambda function errors, execution durations, and API Gateway throttling.
  - Define thresholds for each metric that, when breached, trigger alarms to notify the development team.
  - Integrate alarms with AWS SNS (Simple Notification Service) topics to enable timely notifications via email or other communication channels.
  - Ensure that alarms are configured to cover all essential components, including API Gateway, Lambda functions, and DynamoDB tables.
  - Test alarm configurations to verify that notifications are sent correctly upon threshold breaches.

This development plan systematically breaks down the creation of each CDK stack into manageable, code-focused tasks. Each task is detailed to ensure clarity and facilitate smooth execution by the development team. By adhering to the outlined tasks, the team can efficiently build, integrate, and deploy the chat application within the specified two-week timeframe.

APPENDIX L – Custom development plan generation prompt

```
You are working in a software development agency and a project manager
and software architect approach you telling you that you're assigned to
work on a new project.
You are working on a fully AWS system that will be implemented using AWS
CDK with python, and you need to create a detailed development plan so
that developers can start developing the app.
You will receive the project main description, the project specification
and the selected architecture to this project.

**IMPORTANT**
The architecture is divided into CDK stacks that will be user to build
this application and it is the granularity you must use to create the
development plan.

----- MAIN DESCRIPTION -----
{{ main_description }}
----- END MAIN DESCRIPTION -----
----- PROJECT SPECIFICATIONS -----
{{ project_specs }}
----- END PROJECT SPECIFICATIONS -----
----- ARCHITECTURE -----
{{ architecture }}
----- END ARCHITECTURE -----
----- USE CASES -----
{{ use_cases }}
----- END USE CASES -----


Before we go into the coding part, I want you to split the development of
each CDK stack into smaller tasks so that it is easier to develop, debug
and make the application work.
```

```
Each task needs to be related only to the development of this application
and nothing else - once the application is fully working, that is it.
There shouldn't be a task for researching, deployment, writing
documentation, testing or anything that is not writing the actual code.

**IMPORTANT**
As an experienced tech lead you always follow rules on how to create
tasks. Dividing project into tasks is extremely important job and you
have to do it very carefully.

Now, based on the project details provided, think task by task and create
the entire development plan. Start from the project setup and specify
each task until the moment when the entire app should be fully working
while strictly following these rules:

Here are the revised rules tailored for a tech lead creating a
development plan for a Python AWS CDK (Cloud Development Kit)
application:

**Rule #1**
There should never be a task that is solely for testing or ensuring
something works; every task must involve coding. This should be
considered for every task, especially the final task of the project.
Testing the application will be done as part of each task.

**Rule #2**
This rule applies to the complexity of tasks.
You must ensure the project is not divided into tasks that are too small
or simple without reason, nor too large or complex, making them difficult
to develop, debug, and review. The project will already have the
workspace folder created and basic system dependencies installed, so
tasks for those are unnecessary.
Here are examples of poorly created tasks:

**too simple tasks**
- Set up a Python project and install the AWS CDK.
- Establish an S3 bucket using the default configuration.

**too complex tasks**
```

- Set up a Python project with an AWS CDK application that includes creating a VPC, setting up EC2 instances with specific security groups, creating Lambda functions, integrating with DynamoDB, and implementing a complete CI/CD pipeline.

Avoid creating tasks that are too simple or too complex. Aim for medium complexity tasks. Here are examples of good tasks:

good tasks
- Create a basic CDK stack that provisions a private S3 bucket.
- Implement a Lambda function using the AWS CDK. The Lambda should be triggered by an S3 bucket event and log the event details.

Rule #3
This rule applies to the number of tasks you will create.
Every application should have a different number of tasks depending on its complexity. Think task by task and create the minimum number of tasks that are relevant for this specific application. Here are some examples of apps with different complexity that can give you guidance on how many tasks you should create:

Example #1:
App description: "I want to create a CDK app that provisions an S3 bucket and a Lambda function that logs events."
Number of tasks: 2-3

Example #2:
App description: "Create a CDK app that sets up an API Gateway with Lambda integration, allowing users to make HTTP requests which trigger the Lambda."
Number of tasks: 3-5

Example #3:
App description: "A comprehensive CDK application that provisions a VPC, multiple EC2 instances, a load balancer, and integrates with an RDS database and S3 for storage."
Number of tasks: 6-12

Rule #4

```
This rule applies to writing task 'description'.
Every task must have a clear and very detailed description (minimum of 4
sentences, but can be more). It must be very clear so that even
developers who just moved to this project can execute them without
additional questions. It is not enough to just write something like
"Create an S3 bucket." You have to describe what needs to be done in
detail. Provide as many details as possible and ensure no information is
missing that could be needed for the task.
Here is an example of good and bad task descriptions:

**bad task**
Create a CDK stack for an S3 bucket

**good task**
In 'cdk_stack.py', create a new CDK stack class 'MyS3Stack'. In this
stack, define an S3 bucket with versioning enabled. The bucket should
have a lifecycle rule to transition objects to Glacier after 30 days. Add
the necessary IAM permissions to allow read/write access from a specific
Lambda function.

**Rule #5**
When creating and naming new files, ensure the file naming (camelCase,
kebab-case, underscore_case, etc.) is consistent with the best practices
and coding style of the language.
```